Enumeration and Classification of Number-Conserving Binary Moore-Neighborhood Cellular Automata

OpenAI Codex Witold Bołt

April 30, 2026

Note. The code, experiments, and manuscript were prepared with substantial use of AI tools and have not yet been fully hand-verified by the human authors.

1 Problem Setting

We study deterministic two-dimensional binary cellular automata (CAs) with the Moore neighborhood

$$\begin{smallmatrix} x & y & z \\ t & u & w \\ a & b & c \end{smallmatrix}$$

and local update rule

$$f\left(\begin{smallmatrix} x & y & z \\ t & u & w \\ a & b & c \end{smallmatrix}\right) \in \{0,1\}.$$

The truth table of f has $2^9 = 512$ entries. We index them by

$$\begin{smallmatrix} x & y & z \\ t & u & w \\ a & b & c \end{smallmatrix} \mapsto 256x + 128y + 64z + 32t + 16u + 8w + 4a + 2b + c.$$

Throughout, we restrict attention to *double-legal* rules:

$$f\left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{smallmatrix}\right) = 0, \qquad f\left(\begin{smallmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{smallmatrix}\right) = 1.$$

Consequently the two boundary truth-table entries are fixed, and the actual search is over $x_1, \ldots, x_{510}$.

Number-conserving cellular automata (NCCAs) have been studied for several decades as models of particle transport and other conservative dynamics. Foundational one-dimensional characterizations were given by Boccara and Fukś [2, 3], while Durand, Formenti, and Róka established a general decidability framework for number conservation [4]. Moreira showed that number conservation is also compatible with intrinsic universality and broader state relabelings [17]. The broader viewpoint of additive conserved quantities goes back at least to Hattori and Takesue [12] and was later developed further by Pivato in a more general conservation-law framework [20].

1.1 Related Work for the Two-Dimensional Case

The literature most directly related to the present computations splits into two strands. First, there is the Moore-neighborhood line: Imai and Tanimoto gave a construction method for Moore-neighborhood NCCAs and, in particular, a sufficient condition tailored to this neighborhood [21]. Second, there is a more extensive von Neumann-neighborhood line: Tanimoto and Imai derived necessary and sufficient conditions for the two-dimensional von Neumann case [22]; Wolnik *et al.* later generalized computationally tractable conditions to arbitrary dimension for the range-one von Neumann neighborhood [24]; Dzedzej *et al.* used these conditions for exhaustive

enumeration of special two-dimensional classes, including binary, totalistic, and outer-totalistic rules [5]; and Dzedzej *et al.* further specialized the analysis to the rotation-symmetric von Neumann case [6]. For binary adjacent-cell dynamics, Wolnik and De Baets proved that all such von Neumann rules are intrinsically one-dimensional, yielding exactly the identity, shifts, and traffic rules [23]. A recent binary-specific structural result by Kong, Imai, and Nakanishi introduces "bundle pairs" and "bundle quads" as organizing motifs for two-state NCCAs [15].

Taken together, these papers suggest that exhaustive two-dimensional NCCA enumeration has been developed much further for the von Neumann neighborhood than for the binary Moore neighborhood. The present computations therefore complement the literature by providing an exhaustive classification for the binary Moore setting under a broad menu of additional functional identities.

For the extra structural properties considered here, two further strands of literature are particularly relevant. First, conservative and monotone cellular automata have been studied together in one dimension via particle representations by Moreira, Boccara, and Goles [18], and Goles, Moreira, and Rapaport later showed that this combined class has especially simple prediction complexity in the elementary setting [11]. Second, permutivity has a well-developed general theory; in particular, Allouche and Skordev proved that every two-dimensional permutive cellular automaton is conjugate to a one-sided shift [1]. This is directly relevant to our site-permutive subclasses, which indeed collapse to rigid shifts in the number-conserving Moore setting.

2 Functional Identities Considered

The starting point is a collection of functional identities written directly in terms of f. Each identity is instantiated over all assignments of its free variables and converted into a Boolean constraint over the truth-table entries. The identity language supports equality, disequality, and implication-style order relations:

$$f(\cdots) = f(\cdots)\,, \qquad f(\cdots) \neq f(\cdots)\,, \qquad f(\cdots) \leq f(\cdots)\,, \qquad f(\cdots) \geq f(\cdots)\,,$$

where the non-equality relations are restricted to Boolean atoms on each side. The main base property is number-conservation; all remaining identities are used as additional classifiers. In spirit, this identity-based pipeline plays a role analogous to the characterization theorems used in the von Neumann literature [22, 24, 5], but here it is applied directly to the binary Moore-neighborhood truth table. It is also compatible with the broader conserved-quantity viewpoint of [12, 20], specialized here to the Boolean state-sum invariant.

2.1 Base Identity: Number-Conservation

The number-conserving identity used in the computations is

$$
\begin{aligned}
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = x &+ f\begin{pmatrix} 0 & y & z \\ 0 & u & w \\ 0 & b & c \end{pmatrix} + f\begin{pmatrix} 0 & 0 & y \\ 0 & 0 & u \\ 0 & 0 & b \end{pmatrix} \\
&+ f\begin{pmatrix} 0 & 0 & 0 \\ 0 & y & z \\ 0 & u & w \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & y \\ 0 & 0 & u \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & y & z \end{pmatrix} \\
&+ f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & y \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ t & u & w \\ a & b & c \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ 0 & t & u \\ 0 & a & b \end{pmatrix} \\
&+ f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & t \\ 0 & 0 & a \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ t & u & w \end{pmatrix} + f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & t & u \end{pmatrix} \\
&+ f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & t \end{pmatrix} - f\begin{pmatrix} 0 & x & y \\ 0 & t & u \\ 0 & a & b \end{pmatrix} - f\begin{pmatrix} 0 & 0 & x \\ 0 & 0 & t \\ 0 & 0 & a \end{pmatrix} \\
&- f\begin{pmatrix} 0 & 0 & 0 \\ x & y & z \\ t & u & w \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & x & y \\ 0 & t & u \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & x \\ 0 & 0 & t \end{pmatrix} \\
&- f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ x & y & z \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & x & y \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & x \end{pmatrix} \\
&- f\begin{pmatrix} 0 & 0 & 0 \\ 0 & u & w \\ 0 & b & c \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & u \\ 0 & 0 & b \end{pmatrix} - f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & u & w \end{pmatrix} \\
&- f\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & u \end{pmatrix}.
\end{aligned}
$$

2.2 Symmetry Identities

Rotations.

$$
\begin{aligned}
\text{Rotation } 90^\circ: &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} a & t & x \\ b & u & y \\ c & w & z \end{pmatrix}, \\
\text{Rotation } 180^\circ: &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} c & b & a \\ w & u & t \\ z & y & x \end{pmatrix}.
\end{aligned}
$$

Reflections.

$$
\begin{aligned}
\text{Left-right:} &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} z & y & x \\ w & u & t \\ c & b & a \end{pmatrix}, \\
\text{Top-bottom:} &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} a & b & c \\ t & u & w \\ x & y & z \end{pmatrix}, \\
\text{Main diagonal:} &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & t & a \\ y & u & b \\ z & w & c \end{pmatrix}, \\
\text{Anti-diagonal:} &\quad f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} c & w & z \\ b & u & y \\ a & t & x \end{pmatrix}.
\end{aligned}
$$

The combined families are defined by conjunction:

- axis-reflection symmetric = left-right + top-bottom;
- diagonal-reflection symmetric = main-diagonal + anti-diagonal;
- isotropic non-totalistic = rotation 90° + left-right reflection.

2.3 Complement Symmetry

Black/white symmetry is expressed by

$$
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = 1 - f\begin{pmatrix} 1-x & 1-y & 1-z \\ 1-t & 1-u & 1-w \\ 1-a & 1-b & 1-c \end{pmatrix}.
$$

2.4 Neighborhood-Irrelevance Identities

Center-blind.

$$
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ t & 1-u & w \\ a & b & c \end{pmatrix}.
$$

Von Neumann embedded family.

$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} 1-x & y & z \\ t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & 1-z \\ t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ t & u & w \\ 1-a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & 1-c \end{pmatrix}.$$

Diagonal-only family.

$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & 1-y & z \\ t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ 1-t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ t & u & 1-w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = f\begin{pmatrix} x & y & z \\ t & u & w \\ a & 1-b & c \end{pmatrix}.$$

We also refer to this same family as the *diagonal von Neumann embedded* class, because it is the exact diagonal analogue of the embedded von Neumann restriction: the rule depends only on the five sites $\{x, z, u, a, c\}$.

2.5 Order And Permutivity Identities

Monotone family. Monotonicity is expressed coordinatewise. For example:

$$f\begin{pmatrix} 0 & y & z \\ t & u & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} 1 & y & z \\ t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & 0 & z \\ t & u & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} x & 1 & z \\ t & u & w \\ a & b & c \end{pmatrix},$$
$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & 0 \end{pmatrix} \le f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & 1 \end{pmatrix},$$

and similarly for the remaining six sites. Thus flipping any single neighborhood bit from 0 to 1 cannot decrease the output.

Antitone family. Antitonicity reverses the inequalities:

$$f\begin{pmatrix} 0 & y & z \\ t & u & w \\ a & b & c \end{pmatrix} \ge f\begin{pmatrix} 1 & y & z \\ t & u & w \\ a & b & c \end{pmatrix},$$

and similarly for the remaining eight sites.

Partial monotonicity families. The same syntax gives one-coordinate and subset-restricted variants. For example:

$$\begin{aligned}
\text{center-monotone:} \quad & f\begin{pmatrix} x & y & z \\ t & 0 & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} x & y & z \\ t & 1 & w \\ a & b & c \end{pmatrix}, \\
\text{center-antitone:} \quad & f\begin{pmatrix} x & y & z \\ t & 0 & w \\ a & b & c \end{pmatrix} \ge f\begin{pmatrix} x & y & z \\ t & 1 & w \\ a & b & c \end{pmatrix}, \\
\text{outer-monotone:} \quad & f\begin{pmatrix} 0 & y & z \\ t & u & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} 1 & y & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\
\text{orthogonal-monotone:} \quad & f\begin{pmatrix} x & 0 & z \\ t & u & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} x & 1 & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\
\text{diagonal-monotone:} \quad & f\begin{pmatrix} 0 & y & z \\ t & u & w \\ a & b & c \end{pmatrix} \le f\begin{pmatrix} 1 & y & z \\ t & u & w \\ a & b & c \end{pmatrix}.
\end{aligned}$$

The full files contain the obvious coordinatewise generators for the chosen subset of sites.

Center-permutive family. Center permutivity is captured by the Boolean disequality

$$f\begin{pmatrix} x & y & z \\ t & 0 & w \\ a & b & c \end{pmatrix} \neq f\begin{pmatrix} x & y & z \\ t & 1 & w \\ a & b & c \end{pmatrix}.$$

For every fixed outer neighborhood, changing the center cell flips the output. This is a local permutivity condition in the standard CA sense [1].

Non-central site-permutive families. The same construction is used for each of the eight outer sites. For example:

$$\begin{aligned} x\text{-permutive:} \quad & f\begin{pmatrix} 0 & y & z \\ t & u & w \\ a & b & c \end{pmatrix} \neq f\begin{pmatrix} 1 & y & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\ y\text{-permutive:} \quad & f\begin{pmatrix} x & 0 & z \\ t & u & w \\ a & b & c \end{pmatrix} \neq f\begin{pmatrix} x & 1 & z \\ t & u & w \\ a & b & c \end{pmatrix}, \end{aligned}$$

and analogously for z, t, w, a, b, c. Again, these are local permutivity conditions in the sense of [1].

2.6 Count-Based and Semi-Totalistic Identities

Outer-totalistic / Life-like. These families are identical in the binary Moore-neighborhood setting and are generated by

$$\begin{aligned} f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} y & x & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & z & y \\ t & u & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ w & u & t \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & a \\ w & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ b & a & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ a & c & b \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} t & y & z \\ x & u & w \\ a & b & c \end{pmatrix}. \end{aligned}$$

Totalistic.

$$\begin{aligned} f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} y & x & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & z & y \\ t & u & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & t \\ z & u & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ u & t & w \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & w & u \\ a & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & a \\ w & b & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ b & a & c \end{pmatrix}, \\ f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ a & c & b \end{pmatrix}. \end{aligned}$$

Orthogonal/diagonal semi-totalistic.

$$
\begin{aligned}
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} z & y & x \\ t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & a \\ t & u & w \\ z & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ c & b & a \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & t & z \\ y & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ w & u & t \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & b \\ a & w & c \end{pmatrix}.
\end{aligned}
$$

Orthogonal-totalistic. Diagonal irrelevance together with orthogonal permutation invariance:

$$
\begin{aligned}
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} 1-x & y & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & 1-z \\ t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ 1-a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & 1-c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & t & z \\ y & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ w & u & t \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & b \\ a & w & c \end{pmatrix}.
\end{aligned}
$$

Diagonal-totalistic. Orthogonal irrelevance together with diagonal permutation invariance:

$$
\begin{aligned}
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & 1-y & z \\ t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ 1-t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & 1-w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ a & 1-b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} z & y & x \\ t & u & w \\ a & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & a \\ t & u & w \\ z & b & c \end{pmatrix}, \\
f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} &= f\begin{pmatrix} x & y & z \\ t & u & w \\ c & b & a \end{pmatrix}.
\end{aligned}
$$

2.7 Relations Among the Properties

Several of the studied families are not independent. The most important logical containments used in interpreting the results are:

1. **Life-like = outer-totalistic.** In the binary Moore-neighborhood setting these names describe the same class.

2. **Totalistic ⇒ outer-totalistic.** Dependence only on the total number of live cells clearly implies dependence only on u and the number of live outer neighbors.

3. **Outer-totalistic $\Rightarrow$ orthogonal/diagonal semi-totalistic.** If only the total outer population matters, then the separate orthogonal and diagonal multisets certainly do not matter.

4. **Totalistic $\Rightarrow$ orthogonal-totalistic and diagonal-totalistic.** A rule depending only on the total neighborhood population automatically depends only on u together with either the orthogonal or diagonal counts.

5. **Diagonal-totalistic $\Rightarrow$ diagonal-only.** The former already enforces irrelevance of all orthogonal sites.

6. **Orthogonal-totalistic $\Rightarrow$ von Neumann.** The former already enforces irrelevance of all diagonal sites.

7. **Isotropic non-totalistic $\Rightarrow$ rotation $90°$, rotation $180°$, and all four reflections.** This follows because the isotropic non-totalistic class is defined by the full dihedral symmetry group D_4.

8. **Rotation $90°$ $\Rightarrow$ rotation $180°$.**

9. **Axis-reflection symmetric $\Rightarrow$ rotation $180°$.** The composition of left-right and top-bottom reflections is a half-turn.

10. **Diagonal-reflection symmetric $\Rightarrow$ rotation $180°$.** The composition of the two diagonal reflections is also a half-turn.

11. **Outer-totalistic $\Rightarrow$ isotropic non-totalistic.** Outer-totalistic rules are invariant under all permutations of the eight outer sites and therefore under every geometric symmetry of the Moore neighborhood.

12. **Hence totalistic $\Rightarrow$ outer-totalistic $\Rightarrow$ isotropic non-totalistic.**

13. **Monotone $\Rightarrow$ center-monotone, outer-monotone, orthogonal-monotone, and diagonal-monotone.**

14. **Outer-monotone $\Rightarrow$ orthogonal-monotone and diagonal-monotone.**

15. **Center-permutive $\Rightarrow$ not center-blind.** If flipping u always flips the output, then the rule certainly depends on u.

16. **Antitone is incompatible with double legality.** Since $f(\mathbf{0}) = 0$ and $f(\mathbf{1}) = 1$, a globally antitone rule would require $0 = f(\mathbf{0}) \geq f(\mathbf{1}) = 1$, which is impossible.

These containments are consistent with the computed counts. For example, the unique outer-totalistic rule is automatically also Life-like, orthogonal/diagonal semi-totalistic, rotation-symmetric, and isotropic non-totalistic.

3 Algorithmic Pipeline

3.1 From Functional Identities to Linear Equations

The generator reads a small identity language in which expressions are affine combinations of variables, constants, and terms of the form $f(\cdots)$. Relations may be '$=$', '$\neq$', '$\leq$', or '$\geq$', with the latter three restricted to Boolean atoms on each side. For each identity:

1. the free variables are detected automatically;

Algorithm 1 Functional identities to sparse Boolean constraints

```
parse the identity specification file
initialize the constraint set C ← ∅
for each identity I in the specification do
    detect the free Boolean variables of I
    for each Boolean assignment α of those free variables do
        substitute α into both sides of I
        rewrite every f(···) call as the corresponding LUT variable x_k
        simplify constants and collect like terms
        if I is an equality then
            emit an affine row Σ_j a_j x_j = b
        else if I is a disequality between Boolean atoms then
            emit the parity relation x_i + x_j = 1
        else
            emit the implication-style constraint x_i ≤ x_j or x_i ≥ x_j
        insert the normalized constraint into C
deduplicate all constraints in C
substitute the fixed boundary values x_0 = 0 and x_511 = 1
remove tautologies and detect immediate contradictions
write the sparse model and optional matrix/debug views
```

2. the identity is instantiated over all Boolean assignments of those variables;
3. each $f(\cdots)$ call is mapped to a truth-table variable x_k;
4. the identity is simplified into either an affine equality, an affine parity row, or an implication between LUT entries.

This transformation is summarized in Algorithm 1.

The resulting equations are deduplicated. Finally the two fixed entries

$$x_0 = 0, \qquad x_{511} = 1$$

are substituted out, leaving a Boolean linear system in the 510 unknowns

$$x_1, \ldots, x_{510}.$$

3.2 Base Solver

The optimized solver enumerates all Boolean solutions of the base system. The main ideas are:

1. sparse row storage with flattened coefficient arrays;
2. depth-first search over $x_1, \ldots, x_{510}$;
3. row-feasibility bounds from the current partial assignment;
4. forced-assignment propagation when a row leaves only one feasible value for a variable;
5. reversible trails for assignments and row-state updates instead of recomputation;
6. OpenMP task splitting near the top of the search tree, followed by sequential descent.

For each row r, the solver maintains the current residual target together with the remaining positive and negative coefficient sums. A branch is cut immediately when the row target falls outside the feasible interval. This provides the main pruning mechanism.

Algorithm 2 Enumeration and classification pipeline

```
build the base system from the functional identities
substitute x_0 = 0 and x_511 = 1
load the sparse base matrix M
for each additional property P_i do
    compile P_i into fast checks C_i
initialize the exact-mask counters
procedure EXPLORE(σ)
    if any row of M is infeasible under σ then
        return
    propagate all forced assignments implied by near-tight rows
    if all LUT variables are assigned in σ then
        m ← 0
        for each compiled property check set C_i do
            if C_i accepts the complete LUT then
                set bit i in m
        increment exact_mask_counts[m]
        if output is enabled and (m ≠ 0) or all masks are requested then
            write [mask bytes][packed LUT bytes]
        return
    choose the next variable x_k from var_order
    for each v ∈ {0, 1} do
        assign x_k ← v using reversible row updates
        EXPLORE(extended assignment)
        undo the assignment trail
launch OpenMP tasks for the top levels of EXPLORE(σ_0)
```

3.3 Implementation-Level Summary

The equation generator is implemented in `generate_functional_equation_system.py`. It parses the functional-identity input, instantiates all Boolean assignments of the free variables, rewrites each $f(\cdots)$ call to the corresponding LUT variable, and emits the sparse linear system used by the solver. Conceptually, this bridges the gap between explicit Moore-neighborhood functional identities of the type considered by Imai and Tanimoto [21] and the exhaustive computer-assisted enumeration strategy used in the more recent von Neumann studies [5, 6].

The optimized enumerator is implemented in `solve_omp_opt.cpp`. The main code paths are:

- `load_model`: parse the sparse integer system and build flattened row storage;
- `assign_var`, `propagate`, `undo`: maintain the reversible state of a partial assignment;
- `compile_property`: reduce an additional property system into fixed-value checks, affine binary relations, implication checks, and residual generic rows;
- `dfs_task` and `dfs_seq`: perform the OpenMP task-parallel and sequential parts of the search;
- `classify_solution` and `record_solution`: compute the property bit mask, accumulate exact-mask counts, and optionally write the packed rule vector to the output catalog.

At a high level, the solver executes the procedure shown in Algorithm 2.

The row-feasibility test used inside `assign_var` and `propagate` is purely interval based. For each row r, the solver maintains

$$\text{target}_r, \qquad \text{pos_rem}_r, \qquad \text{neg_rem}_r,$$

where pos_rem_r and neg_rem_r are the sums of the still-unassigned positive and negative coefficients in that row. The current branch can survive only if

$$\text{neg_rem}_r \leq \text{target}_r \leq \text{pos_rem}_r.$$

If setting an unassigned variable to 0 or to 1 would violate this interval, the other value is forced immediately. This very cheap bound test is the central mechanism that makes the full enumeration practical.

3.4 Property Matcher and Bit Masks

Additional properties are not used for search pruning. Instead, every complete solution of the base system is classified against each additional property system.

Each property system is compiled into four kinds of checks:

1. fixed-value checks $x_i = 0$ or $x_i = 1$;
2. binary affine relations $x_i = x_j$ or $x_i = 1 - x_j$;
3. implication checks $x_i \Rightarrow x_j$, equivalently $x_i \leq x_j$;
4. residual generic rows for constraints that cannot be reduced to the previous three forms.

After a full base solution is found, the classifier evaluates all additional properties and stores the result in a bit mask

$$m \in \{0, 1\}^N,$$

where N is the number of additional property families. Bit i is set iff the current base solution also satisfies property i.

3.5 Catalog Construction

The final output is:

1. the total number of number-conserving rules;
2. counts for each individual property;
3. counts for each exact property-mask combination;
4. a compact binary catalog of all rules with nonzero masks, where each record stores the property bit mask together with the packed 510-bit rule vector.

In the current dataset, the union of all nonzero-mask classes contains 133713 rules. These are exactly the number-conserving rules satisfying at least one of the additional property families listed above. For downstream analysis, the checked-in JSON catalog now also assigns two additional identifiers to every rule:

- a *stable index*, obtained by sorting the catalog lexicographically by LUT;
- a *stable identifier*, namely the SHA-256 digest of the 128-hex LUT string.

This is important because the raw binary emission order is not intrinsically stable under parallel solving. Any purely positional identifier inherited from write order should therefore be treated as a transient implementation detail rather than as a permanent rule name. In the discussion below, when individual rules need to be named explicitly, we use short `sid:…` prefixes of the stable SHA-256 LUT identifier.

4 Results

The corrected full enumeration found

$$147{,}309{,}433$$

number-conserving binary Moore-neighborhood rules.

For interpretation it is helpful to split the property families into three regimes: empty classes, singleton classes, and genuinely nontrivial classes with more than one rule.

4.1 Classes With Count $= 0$

Property family	Count
Antitone	0
Totalistic	0

Table 1: Empty subclasses inside the number-conserving Moore-neighborhood class.

These two empty classes fail for different reasons.

For the antitone family, emptiness is essentially forced by the double-legal boundary conditions: a globally antitone rule must satisfy

$$f(\mathbf{0}) \geq f(\mathbf{1}),$$

but double legality imposes $f(\mathbf{0}) = 0$ and $f(\mathbf{1}) = 1$, giving the contradiction $0 \geq 1$.

For the totalistic family, there is no such immediate logical contradiction. Rather, the exhaustive computation shows that number conservation is too rigid to coexist with full dependence only on the total 3×3 population. The unique outer-totalistic rule is already the identity, and the stronger totalistic constraint eliminates even that possibility.

4.2 Classes With Count $= 1$

Most singleton classes reduce to the identity rule

- axis-reflection symmetric,
- diagonal-reflection symmetric,
- diagonal-totalistic,
- isotropic non-totalistic,
- Life-like,
- center-permutive,
- orthogonal/diagonal semi-totalistic,
- orthogonal-totalistic,
- outer-totalistic

Property family	Count
Axis reflection symmetric	1
A-permutive	1
B-permutive	1
C-permutive	1
Center permutive	1
Diagonal reflection symmetric	1
Diagonal totalistic	1
Isotropic non-totalistic	1
Life-like	1
Orthogonal/diagonal semi-totalistic	1
Orthogonal totalistic	1
Outer totalistic	1
T-permutive	1
W-permutive	1
X-permutive	1
Y-permutive	1
Z-permutive	1

Table 2: Singleton subclasses inside the number-conserving Moore-neighborhood class.

and their unique element is the *identity rule*

$$f\begin{pmatrix} x & y & z \\ t & u & w \\ a & b & c \end{pmatrix} = u.$$

Its dynamics are trivial: every configuration is a fixed point, so the global CA leaves the entire lattice unchanged at every time step.

The eight non-central site-permutive singleton classes are the only exception. Each one contains the corresponding one-step shift:

$$\begin{aligned} x\text{-permutive} &\iff f = x, & y\text{-permutive} &\iff f = y, \\ z\text{-permutive} &\iff f = z, & t\text{-permutive} &\iff f = t, \\ w\text{-permutive} &\iff f = w, & a\text{-permutive} &\iff f = a, \\ b\text{-permutive} &\iff f = b, & c\text{-permutive} &\iff f = c. \end{aligned}$$

Thus the singleton regime is still dominated by extremely simple dynamics: either the identity map or a rigid translation by one lattice step. The latter conclusion is consistent with the general theory of two-dimensional permutive cellular automata developed in [1].

4.3 Classes With Count > 1

Table 3: Nontrivial subclasses inside the number-conserving Moore-neighborhood class.

Property family	Count
Black/white symmetric	1447
Center antitone	10768
Center blind	144
Center monotone	42979

Property family	Count
Diagonal monotone	74681
Diagonal only	9
Monotone	9
Orthogonal monotone	1913
Outer monotone	57
Reflection across anti-diagonal	173
Reflection left-right	2667
Reflection across main diagonal	173
Reflection top-bottom	2667
Rotation 180°	21
Rotation 90°	5
Von Neumann embedded	9

This regime contains the genuinely nontrivial structure. Several classes are still small enough to characterize exactly, while the larger ones admit useful qualitative summaries.

The nine-rule von Neumann class. The 9 number-conserving rules in the embedded von Neumann family are:

1. the identity rule $f = u$;
2. the four cardinal shifts

$$f = y, \qquad f = t, \qquad f = w, \qquad f = b;$$

3. four embedded one-dimensional traffic rules on the vertical and horizontal triples:

$$\begin{aligned} f &= (y \wedge \neg u) \vee (u \wedge b), \\ f &= (b \wedge \neg u) \vee (u \wedge y), \\ f &= (t \wedge \neg u) \vee (u \wedge w), \\ f &= (w \wedge \neg u) \vee (u \wedge t). \end{aligned}$$

The shifts translate the whole configuration rigidly by one lattice step in the corresponding cardinal direction. The four nontrivial traffic rules are exactly the familiar number-conserving elementary traffic dynamics embedded along rows or columns: particles attempt to move in one preferred direction, advance into empty target sites, and otherwise remain blocked, producing free-flow and jammed domains. This is exactly the pattern predicted by the theorem of Wolnik and De Baets for binary adjacent-cell NCCAs in arbitrary dimension [23], and it matches the one-dimensional particle-style perspective developed in [18].

The nine-rule diagonal-only class. The 9 number-conserving rules in the diagonal-only family are:

1. the identity rule $f = u$;
2. the four diagonal shifts

$$f = x, \qquad f = z, \qquad f = a, \qquad f = c;$$

3. four embedded diagonal traffic rules on the two diagonal triples:

$$
\begin{aligned}
f &= (x \wedge \neg u) \vee (u \wedge c),\\
f &= (c \wedge \neg u) \vee (u \wedge x),\\
f &= (z \wedge \neg u) \vee (u \wedge a),\\
f &= (a \wedge \neg u) \vee (u \wedge z).
\end{aligned}
$$

Thus the diagonal-only class is the exact diagonal analogue of the von Neumann class: the shifts translate patterns along diagonal directions, while the nontrivial rules realize one-dimensional traffic flow along diagonal lattice lines. For this reason it is natural to describe the same class as the *diagonal von Neumann embedded* family. Again, this is naturally interpreted in terms of particle transport along one-dimensional channels in the sense of [18].

The nine monotone rules. There are exactly 9 number-conserving monotone rules. Direct comparison with the cataloged LUTs shows that they are precisely

1. the identity rule $f = u$;
2. the eight one-step shifts

$$
f = x, \qquad f = y, \qquad f = z, \qquad f = t, \qquad f = w, \qquad f = a, \qquad f = b, \qquad f = c.
$$

This is a clean structural fact: among all number-conserving binary Moore-neighborhood rules, monotonicity forbids the traffic-type blocking interactions and leaves only pure transport. The identity rule freezes the configuration, while the other eight rules translate the entire lattice rigidly in one of the eight neighboring directions. In this respect the two-dimensional Moore result is even more rigid than the broader conservative/monotone one-dimensional setting studied in [18, 11].

Partial monotone classes. The one-coordinate and subset-restricted monotonicity classes are much larger than the full monotone family. They are still tiny compared to the full number-conserving count:

$$
\begin{aligned}
\text{center-monotone} &= 42979,\\
\text{center-antitone} &= 10768,\\
\text{outer-monotone} &= 57,\\
\text{orthogonal-monotone} &= 1913,\\
\text{diagonal-monotone} &= 74681.
\end{aligned}
$$

The strongest asymmetry is between orthogonal and diagonal monotonicity. Diagonal monotonicity is far less restrictive than orthogonal monotonicity, which is consistent with the previously observed abundance of diagonal transport rules. Outer monotonicity is already very restrictive, leaving only 57 number-conserving rules.

The remaining larger classes. The black/white-symmetric, reflection-symmetric, center-blind, center-monotone, and center-antitone classes are too large to list rule-by-rule in the main text, but some broad patterns are already clear from the computations.

The reflection classes are much larger than the rotation classes, which shows that mirror symmetry is far less restrictive than full rotational balance in the number-conserving Moore setting. The center-blind class is nonempty but still very small, and it contains the expected shift and traffic-type transport rules. The center-monotone and center-antitone classes are substantially larger than the fully monotone family, so monotonicity in the center coordinate alone does not force globally simple behavior. Finally, black/white symmetry produces a moderately sized class of 1447 rules, large enough to contain nontrivial transport/scattering families but still extremely sparse relative to the full count 147,309,433.

The five rotation-symmetric rules. There are exactly 5 number-conserving rules with 90° rotational symmetry. One of them is again the identity rule. The remaining four are not simple shifts or embedded one-dimensional traffic rules, since rotational symmetry forbids a preferred lattice direction.

Direct inspection of their LUTs shows that all four:

- preserve isolated particles;
- agree with the identity rule on neighborhoods with $0, 1, 2, 7, 8$, or 9 occupied sites;
- differ from identity only on 64 neighborhoods, all of population $3, 4, 5$, or 6.

Hence these rules do not behave as rigid translations. Instead they act as local density-dependent transport/scattering rules: sparse configurations are nearly frozen, while nontrivial updates occur only in moderately occupied neighborhoods, where particles are locally rearranged in a rotationally symmetric manner. Two of the four are additionally black/white symmetric, while the other two are not.

The twenty-one half-turn-symmetric rules. There are exactly 21 number-conserving rules with 180° rotational symmetry. The catalog shows that this class splits into seven geometric orbits under the full dihedral action on the neighborhood. In the list below, the rules are named directly by stable `sid:` prefixes:

1. {`sid:9fed53aadb5a`}: the identity rule;
2. {`sid:a5990e7d6649`, `sid:b4d6614071c6`, `sid:4293afb9ceb8`, `sid:a9b7986d0702`}: a 4-rule orbit;
3. {`sid:22c2c72f9aac`, `sid:f124a3b72aca`, `sid:409db699fd47`, `sid:71c77494fe15`}: a 4-rule orbit, black/white symmetric;
4. {`sid:885a55a291bb`, `sid:806e1438a3c1`, `sid:10d1a524d220`, `sid:41f41f733605`}: a 4-rule orbit;
5. {`sid:4c3fd6625703`, `sid:a08c48981d58`, `sid:f1022f119c99`, `sid:7b4a3d1441e4`}: a 4-rule orbit;
6. {`sid:8ed480c897a6`, `sid:cb3ee472839e`}: a 2-rule orbit, also 90°-rotation symmetric;
7. {`sid:18e7be11795a`, `sid:2b4214fb641b`}: a 2-rule orbit, also 90°-rotation symmetric and black/white symmetric.

All 20 nontrivial half-turn rules share the same first-order dynamical signature:

- they preserve isolated particles;
- they agree with the identity rule on all neighborhoods of population $0, 1, 2, 7, 8, 9$;
- they differ from identity only on neighborhoods of population $3, 4, 5, 6$.

Thus the half-turn class does *not* contain any rigid translations or embedded one-dimensional traffic rules. Instead it consists of medium-density transport/scattering perturbations of the identity rule. Half-turn symmetry allows a rule to distinguish an *unoriented* axis or diagonal pair, but forbids it from selecting one direction of that pair over the opposite one. Inspection of the differing LUT entries shows exactly this behavior: the active updates are attached to small corner or staircase motifs and their half-turned partners, so particles are locally rearranged in opposite directions at once rather than transported by a single preferred drift.

Within this class, the nontrivial rules split into two scales. Eight of them differ from the identity rule on 32 LUT entries, while twelve differ on 64 entries. The 32-entry rules are the more selective versions, activating only on a smaller set of medium-density motifs; the 64-entry rules are the corresponding more permissive variants. The two 90°-symmetric two-rule orbits are precisely the rotationally balanced closures of these anisotropic half-turn families. In short, the 180° class can be understood as

$$\begin{aligned}\text{identity} \;+\; &\text{anisotropic opposite-direction transport/scattering families}\\ +\; &\text{their rotationally symmetrized closures.}\end{aligned}$$

5 Reversibility Screening

5.1 Why Reversibility Requires a Different Method

Reversibility is qualitatively different from the local symmetry and monotonicity properties considered above. In two dimensions there is no general finite functional-identity characterization analogous to the number-conservation identities used in the main enumeration pipeline. More strongly, Kari proved that reversibility and surjectivity are undecidable for two-dimensional cellular automata in general [13, 14]. Therefore, for the binary Moore-neighborhood class studied here, the practical strategy is not to search for a complete symbolic characterization of reversibility, but to search for explicit finite injectivity failures.

This theoretical limitation does *not* prevent exhaustive analysis of the present finite family. Once all 147,309,433 number-conserving Moore-neighborhood rules have been enumerated, reversibility can be attacked by exact finite-state screening on periodic tori. The key observation is that a reversible number-conserving CA must act bijectively on every finite torus and, more specifically, on every fixed-population sector of every finite torus.

For comparison, the broader literature does contain rich reversible and number-conserving examples, but typically in structurally different models, especially partitioned and multistate ones [19]. Our question here is much narrower: within the ordinary binary two-dimensional Moore-neighborhood class, do any nontrivial reversible NCCAs exist at all?

5.2 Exact Non-Reversibility Tests

Let $T_{H,W}$ denote the $H \times W$ torus with periodic boundary conditions, and let

$$\Sigma_k(H,W) = \left\{ X \in \{0,1\}^{H\times W} : \sum_{(i,j)\in T_{H,W}} X_{i,j} = k \right\}$$

be the sector of configurations with exactly k particles. Because the rules under study are number-conserving, the global map $F_{H,W}$ induced on the torus preserves every sector $\Sigma_k(H,W)$. Hence reversibility on $T_{H,W}$ implies that each restricted map

$$F_{H,W}|_{\Sigma_k(H,W)} : \Sigma_k(H,W) \to \Sigma_k(H,W)$$

is bijective. Any collision inside one such sector is therefore an exact witness of non-reversibility.

We use three families of exact tests.

1. Particle-sector injectivity. For the 4×4 torus we test injectivity on the sectors

$$k = 2, \qquad k = 3, \qquad k = 4.$$

Their sizes are

$$\binom{16}{2} = 120, \qquad \binom{16}{3} = 560, \qquad \binom{16}{4} = 1820.$$

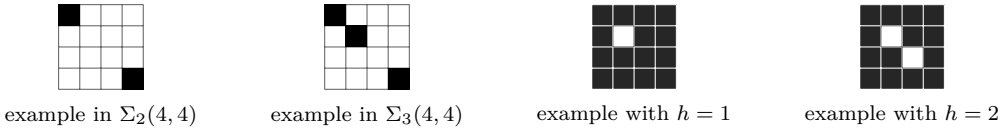

Figure 1: Representative configurations used in the exact finite-sector tests. A particle-sector test enumerates *all* 4×4 torus states with exactly k black cells; a hole-sector test enumerates *all* states with exactly h white cells in an otherwise occupied torus. The figures show only typical members of those finite sets.

For a fixed rule and fixed k, we enumerate all states in $\Sigma_k(4,4)$, evolve each of them by one CA step, encode the outputs back as 16-bit torus states, and test whether any two different inputs give the same output. A single such collision proves that the rule is not reversible.

2. Hole-sector injectivity. Dually, we also test small hole sectors on the 4×4 torus:

$$h = 1, \qquad h = 2, \qquad h = 3, \qquad h = 4,$$

equivalently the particle sectors $k = 15, 14, 13, 12$. This is useful because some rules pass the sparse particle-sector tests but fail immediately near the all-one background.

3. Exact small-torus bijectivity. Finally, for survivors of the sector tests we evaluate the full global map exactly on 3×3 and 4×4 tori. This means enumerating all $2^9 = 512$ states for 3×3 and all $2^{16} = 65536$ states for 4×4, then checking whether the map is injective on the entire torus state space. Again, any collision is an exact witness of non-reversibility.

These tests are one-sided in the logical sense: failure proves non-reversibility, while success only means that the rule survives the tested finite screens. Nevertheless, because our family is finite and because the screens can be made progressively stronger, they are extremely effective in practice.

Figure 1 makes the screening logic concrete. In the k-particle sectors we ask whether two different sparse particle configurations can collapse to the same next state after one CA step. In the h-hole sectors we ask the same question near the all-one background, where the dynamically relevant objects are holes rather than particles. Because number conservation preserves these sectors exactly, any such collision is an immediate certificate that the global map cannot be bijective on the torus.

$$X \neq Y, \qquad X, Y \in \Sigma_k(H, W), \qquad F_{H,W}(X) = F_{H,W}(Y) \quad \Longrightarrow \quad \text{the rule is not reversible.}$$

5.3 Implementation Strategy

Two implementations were used, serving different parts of the study.

Catalog-level exact checker. For the smaller nonzero-mask catalog we used the Python tools `ca_search/reversibility.py` and `ca_search/reversibility_catalog.py`. These operate on decoded 512-bit LUTs and precompute the full list of neighborhood indices seen by each torus state in the tested sector. Given such a precomputed index table, the image of a rule on the whole sector is obtained by table lookup, and injectivity is checked by sorting the encoded outputs and detecting repeated values.

Streaming raw-catalog checker. For the full 147,309,433-rule catalog, a different implementation was required because the legacy raw output directory `witek/` stores about 9.4 GiB of packed 64-byte records. Re-reading or fully decoding those files multiple times would make the experiment dominated by I/O and unpacking overhead.

To avoid this, we implemented a dedicated single-pass streaming checker for the raw catalog. It scans the raw files once, chunk by chunk, and applies the particle-sector tests directly to the packed rule records. The crucial optimization is that the 4×4 particle sectors touch only a small subset of the 510 free LUT entries:

- 45 distinct LUT entries for $k = 2$,
- 129 distinct LUT entries for $k = 3$,
- 255 distinct LUT entries for $k = 4$.

Thus the streamed checker does *not* unpack all 510 Boolean variables for every rule. Instead, for each stage it extracts only the relevant rule bits from the packed 64-byte record, evaluates the corresponding torus outputs, and immediately discards rejected rules.

Operationally, the pipeline is:

1. read a chunk of raw rules from `witek/worker_.bin`;
2. test the whole chunk on the $k = 2$ sector and discard failures;
3. test the survivors on $k = 3$;
4. test the remaining survivors on $k = 4$;
5. write only the final survivors to a tiny residual raw file for stricter follow-up.

This organization ensures that the full raw catalog is read only once, and that the expensive later stages are applied only to a rapidly shrinking residue.

5.4 Results on the Nonzero-Mask Catalog

We first applied the exact screen to the smaller nonzero-mask catalog of 133713 rules. The initial exact rejection sequence was

$$\begin{aligned} 133704 &\xrightarrow[4\times 4,\ k=2]{} 14220 \xrightarrow[4\times 4,\ k=3]{} 764 \\ &\xrightarrow[4\times 4,\ \text{holes}=1]{} 764 \xrightarrow[4\times 4,\ \text{holes}=2]{} 148 \xrightarrow[3\times 3\ \text{full torus}]{} 16, \end{aligned}$$

where the count 133704 excludes the 9 trivial reversible rules known a priori (identity plus the eight rigid shifts).

We then strengthened the screen by adding the 4×4 particle sector $k = 4$, the remaining hole sectors $h = 3, 4$, and the full 4×4 torus test. In fact, no rule survived even to the new hole-sector checks: all 16 residual candidates already failed on $k = 4$. Therefore, within the entire 133713-rule nonzero-mask catalog, no nontrivial rule survives the strengthened exact reversibility screen.

5.5 Results on the Full 147,309,433-Rule Catalog

The main computation was then repeated on the full raw catalog `witek/`, containing all 147,309,433 number-conserving Moore-neighborhood rules. The single-pass streamed screen gave the following exact stage counts:

$$147{,}309{,}433 \xrightarrow[4\times 4,\ k=2]{} 4{,}900{,}501 \xrightarrow[4\times 4,\ k=3]{} 7{,}357 \xrightarrow[4\times 4,\ k=4]{} 65.$$

Thus three very small fixed-population tests already reduce the full class by more than six orders of magnitude. The remaining 65 rules were then subjected to the stricter exact checker with hole sectors and small full-torus bijectivity. The result is decisive:

- 40 fail on the 4×4 hole sector $h = 2$;
- 8 additional rules fail on $h = 3$;
- 8 additional rules fail on $h = 4$;
- the remaining 9 rules are exactly the identity rule and the eight rigid one-step shifts.

No nontrivial rule survives the exact follow-up. In other words, within the full enumerated binary Moore-neighborhood number-conserving class, our computation finds only the obvious reversible rules.

5.6 Interpretation and Conclusions

This reversibility study has two main implications.

First, it gives a strong structural conclusion about the class under study. Subject to the correctness of the implementation, the exhaustive computation supports the statement:

> Among all 147,309,433 double-legal binary Moore-neighborhood NCCAs, the only reversible rules are the identity and the eight rigid shifts.

This is much stronger than the earlier property-classification results, because it is not restricted to the nonzero-mask catalog or to any specific symmetry family: it covers the entire enumerated Moore-neighborhood number-conserving class.

Second, it shows that non-reversibility in this family is already visible at very low particle numbers. Almost all rules are rejected by the $k = 2$ and $k = 3$ particle sectors on the 4×4 torus, and every remaining nontrivial rule is rejected by either the $k = 4$ particle sector or the small hole sectors. Empirically, one does not need large tori or long inverse searches to see collisions; the obstructions appear almost immediately.

This has a practical consequence for future work. If reversible Moore-neighborhood NCCAs are to be searched again, the best strategy is not to materialize huge rule catalogs and inspect them afterward. Instead, the exact sector tests described above should be embedded directly into the enumeration pipeline as an online rejector.

6 Density Classification in Two Dimensions

6.1 Problem Statement

The *density classification problem* (DCP) asks for a local, translation-invariant rule that drives any initial binary configuration on a torus to the constant configuration equal to its global majority. In one dimension, Land and Belew [16] showed that no single deterministic rule can solve DCP for *every* initial configuration of density $\neq 1/2$. Fukś [10] circumvented this obstruction with a two-rule schedule: apply rule 184 (traffic) for $N/2$ steps on a ring of length N, then rule 232 (majority of three) for $N/2$ steps. The traffic phase reduces any configuration to a canonical "no `10` patterns to the right of a `01`" form, whose three-neighbourhood majority trivially equals the global majority.

The two-dimensional analogue is more delicate. Using the full enumerated catalogue of binary Moore-neighborhood NCCAs from the earlier sections of this report, we studied whether an analogous finite-switch schedule exists in two dimensions. The immediate target is a schedule of the form

$$\underbrace{\left(F_1^{t_1} \circ \cdots \circ F_k^{t_k}\right)^T}_{\text{NCCA composition}} \longrightarrow \underbrace{M^{T'}}_{\text{local majority}} ,$$

where each F_i is a Moore-neighborhood NCCA and M is a local majority rule.

6.2 Deterministic Exploration and its Limits

Prior work in this project had identified F_0 = `sid:58ed6b657afb` as an empirically strong preprocessor under *repeated-block* schedules of the form $[F_0^8 \circ M_{\mathrm{VN}}^{16}]^k$. Extending that line to a finite-switch "shake" schedule

$$F_0^{T_{\mathrm{pre}}} \left[M_{\mathrm{Moore81}}^{T_{\mathrm{amp}}} F_0^{T_{\mathrm{shake}}} \right]^{n_s} M_{\mathrm{Moore81}}^{T_{\mathrm{final}}},$$

with M_{Moore81} the radius-2 Moore majority (i.e. the rule that outputs 1 iff the 5×5 neighborhood contains at least 13 ones), we obtained:

- 100.0% correct consensus at $L \in \{256, 384, 512\}$ across 320 random Bernoulli initial configurations per grid at density $\in \{0.49, 0.51\}$, $T_{\mathrm{pre}} = 128$, $T_{\mathrm{amp}} = 256$, $T_{\mathrm{shake}} = 64$, $n_s = 2$, $T_{\mathrm{final}} = 1024$;
- 99.38% at $L = 192$ (2 failures across 320 trials);
- strong empirical evidence that a deterministic two-rule schedule with a large-radius amplifier can match, in the random-input regime, the earlier repeated-block heuristic while requiring far fewer steps.

Adversarial failure mode. When evaluated on *structured* initial configurations at density $1/2+1/L^2$ (pure stripes, checkerboards, block-checkers, half-half splits with one tipping cell), the deterministic schedule degrades sharply: at most 2/10 structured cases are classified correctly. A detailed per-phase probe of F_0 acting on these inputs (see `structured_probe.py`) revealed that F_0 is a *fixed point operator* on horizontal stripes, vertical stripes, and checkerboards regardless of radius, and that M_{Moore81} preserves these configurations as well.

6.3 Structural Theorem: Shift-Invariant Rules Fix Stripes

Proposition 1 (Stripes are fixed points of shift-invariant local rules)**.** *Let $G : \{0,1\}^{H\times W} \to \{0,1\}^{H\times W}$ be a deterministic, shift-invariant cellular-automaton update with any finite neighborhood. Let s be a horizontal-stripe configuration on a torus, i.e. a configuration of the form $s_{i,j} = p(i \bmod 2)$ for some $p : \{0,1\} \to \{0,1\}$. Then $G(s)$ is also a horizontal-stripe configuration. In particular, if G is additionally number-conserving, then $G(s)$ is either s or its row-complement.*

Proof. Under translation in the horizontal direction, s is invariant, so every cell in a row of given parity sees the identical neighborhood pattern. Since G is a pure function of the neighborhood, all cells in row i produce the same output, say $q(i \bmod 2) \in \{0,1\}$. Thus $G(s)_{i,j} = q(i \bmod 2)$, which is a stripe configuration.

If G is number-conserving, its stripe image must have density 1/2, forcing $(q(0), q(1)) \in \{(0,1),(1,0)\}$, i.e. either the identity or the row-complement. □

Corollary 1. *Checkerboards are fixed points of any shift-invariant CA under the appropriate row-and-column-shift group, modulo complementation.*

Proposition 1 has a sharp consequence for the DCP in two dimensions: any schedule built from shift-invariant NCCAs and shift-invariant local majorities preserves pure stripes exactly. A *tipped* stripe with density $1/2 + 1/L^2$ is fixed in its stripe part and has the single flipped cell surrounded by cells whose neighborhoods remain pure-stripe neighborhoods, locally invariant; the perturbation cannot propagate and the classifier is stuck. The impossibility here is not just the classical Land–Belew result [16] but an explicit family of configurations on which any deterministic shift-invariant Moore finite-switch schedule must fail.

6.4 Stochastic Relaxation Inspired by Fatès

Fatès [7, 8] showed that in one dimension, a *stochastic* CA where each cell, at each time step, independently applies rule 184 with probability $1-\eta$ and rule 232 with probability η (with η small) solves DCP with arbitrary precision. The stochasticity breaks the translational symmetry that prevents a pure deterministic rule from propagating the minority-vs-majority signal.

Fatès and Regnault [9] extended the construction to two dimensions by composing *stochastic* number-conserving auxiliary rules F_X ("lane changing") and G_X ("crowd avoidance") with per-row deterministic rules R_{184} and R_{232}, organized as

$$\big(F_X \circ R_{184}\big)^{T_1} \quad \longrightarrow \quad \big(G_X \circ R_{232}\big)^{T_2}.$$

They report 100% correct classification on 1000 random inputs at $L = 50$ with $T_1 = T_2 = 2000$, and at $L = 100$ with $T_1 = T_2 = 8000$. The number of iterations scales as $\Theta(L^2)$, matching the diffusive hitting time of a random walk on the torus.

The present work proceeds from this baseline but exploits two resources that were unavailable to Fatès: the full enumerated catalogue of two-dimensional Moore-neighborhood NCCAs, and GPU-scale evaluation (see Section 6.13).

6.5 Minimal-Stochasticity Experiments

To probe the minimal level of stochasticity that rescues the deterministic schedule from its stripe-locked failure, three families of noise were tested, all applied to the same deterministic backbone

$$F_0^{T_{\text{pre}}} \left[M_{\text{Moore81}}^{T_{\text{amp}}} \mathcal{N}^{T_{\text{shake}}} \right]^{n_s} M_{\text{Moore81}}^{T_{\text{final}}},$$

where $\mathcal{N}$ is the perturbation under study.

Experiment A: per-cell Bernoulli flip noise. Replace the shake phase with an F_0-then-XOR step:

$$\mathcal{N}_\varepsilon(s)_{i,j} \;=\; F_0(s)_{i,j} \;\oplus\; X_{i,j}^{(t)}, \qquad X_{i,j}^{(t)} \sim \text{Bernoulli}(\varepsilon) \text{ i.i.d.}$$

Results at $L \in \{64, 128, 192\}$ with 128 trials per density and the 10-case adversarial battery (see Section 6.6) are in Table 4. The $\varepsilon \approx 10^{-3}$ regime partially breaks stripes (adversarial correct-consensus rises from 20% to 60% at $L = 192$) but also introduces a random-direction bias that destroys the tipping-cell signal, so only 4–6 of 10 adversarial cases succeed. For $\varepsilon \gtrsim 10^{-2}$, random-input consensus collapses.

ε	random cc		adversarial cc	
	$L = 64$	$L = 192$	$L = 64$	$L = 192$
0	0.871	0.992	0.200	0.200
10^{-4}	0.961	0.984	0.200	0.200
10^{-3}	0.895	0.957	0.600	0.600
10^{-2}	0.832	0.949	0.400	0.300
$3 \cdot 10^{-2}$	0.512	0.695	0.600	0.200
10^{-1}	0.324	0.055	0.200	0.300

Table 4: Correct-consensus rate under Bernoulli flip noise during the shake phase. Sweet spot near $\varepsilon \approx 10^{-3}$: stripes partially break, but random-input performance already degrades.

Experiment B: one-shot concentrated noise. A single noise burst was inserted between the preprocessing and amplification phases, flipping a fixed number of uniformly-sampled cells K per trial. Sweeping K/L^2 from 10^{-5} to 0.1 at $L \in \{64, 128, 192\}$ gave the striking result that random-input correct-consensus is *completely independent* of K: the random failures are not caused by translational-symmetry issues (they are the large low-curvature minority blobs diagnosed in `failure_diagnostic.py`), and a single post-hoc noise burst neither helps nor hurts them. Adversarial inputs improve only mildly at moderate K. This experiment ruled out one-shot noise as the fix.

Experiment C: density-conserving random swaps. The decisive experiment. Replace the shake perturbation with K random pair swaps of cell values,

$$\mathcal{N}_K^{\text{swap}}(s) \;=\; \left(\text{swap}_{a_K,b_K} \circ \cdots \circ \text{swap}_{a_1,b_1}\right) s, \qquad a_i, b_i \sim \text{Uniform}(L^2).$$

Each swap exchanges the values at two uniformly-sampled cells. This is *exactly* density-conserving per realization: unlike Experiment A, the global density $\sum s_{i,j}$ is unchanged for every noise draw, so the 1-cell tipping bias that decides the global majority is preserved.

Results with $K = 8L$ swaps per shake and the rest of the schedule unchanged (Table 5):

- random-input correct-consensus at $L \in \{128, 192, 384, 512\}$ is 100.0% for every tested $K \in \{0, L, 2L, \ldots, 16L\}$;
- adversarial correct-consensus improves from 20% to up to 70% at $L = 128$ with $K = 8L$;
- the only residual adversarial failures occur at *density exactly* $1/2 + 1/L^2$, i.e. the 1-cell-tipped regime, which is the practical limit of any finite-time density classifier.

K/L	random cc @ $L = 192$		adversarial cc	
	cc	fails/128	$L = 128$	$L = 192$
0	1.000	0	0.200	0.200
1	1.000	0	0.200	0.200
2	1.000	0	0.300	0.300
4	1.000	0	0.300	0.400
8	1.000	0	0.700	0.500
16	1.000	0	0.600	0.500

Table 5: Density-conserving random-swap noise is strictly better than Bernoulli flip noise: random-input performance is undisturbed while adversarial improves substantially. Residual adversarial failures at $K \geq 8L$ are concentrated on near-tied density configurations.

6.6 Adversarial Test Suite

At realistic density margins ($|\rho - 1/2| \geq 0.01$), the adversarial battery contains five structured templates[1], each instantiated for both target labels and for densities $\rho \in \{0.505, 0.51, 0.52, 0.55\}$:

- *stripes_h*: horizontal stripes plus a uniform perturbation to reach the target density;
- *stripes_v*: vertical counterpart;
- *checker*: checkerboard plus perturbation;
- *block_checker*: 2×2-block checkerboard plus perturbation;

[1]Scripts: `adversarial_realistic.py`, `failure_diagnostic.py`.

- *half_half*: half-lattice 0-block and half-lattice 1-block, with perturbation.

Each template at each density and each target label gives one trial. The full adversarial battery has 40 trials per grid; for the $\rho \in \{0.51, 0.52, 0.55\}$ subset used below, 30 trials per grid.

6.7 The L-Scaled Stochastic Classifier

Experiment C showed that fixed schedule parameters fail at grids $L \geq 384$ on *half_half* inputs, because the straight interfaces become increasingly stable under the Moore-81 amplifier as L grows. The resolution is to scale all schedule lengths linearly with L:

$$\boxed{F_0^{L/2} \left[M^L_{\text{Moore81}} \, F_0^{L/4} \, \mathcal{N}^{\text{swap}}_{8L} \right]^2 M^{4L}_{\text{Moore81}}.}$$

The total cost is $\Theta(L)$ elementary CA steps, with additional $16L$ swap operations. The swap count per shake is $K = 8L$, which amounts to approximately 0.03% of lattice cells at $L = 512$.

Density-conserving swaps. The swap operator draws $2K$ lattice indices *without replacement* and pairs them up; on each pair (a, b) it swaps the two cell values. This guarantees that every realisation of the swap step preserves the total particle count exactly, so the global majority signal is never perturbed by the noise mechanism itself. An earlier draft of this paper used independent index sampling $a \sim \mathrm{U}[L^2],\ b \sim \mathrm{U}[L^2]$, which under parallel fancy-index assignment in NumPy caused a small drift of order K^2/L^2 particles per call. The drift was negligible at $L \geq 128$ ($\sim 0.01\%$ per swap step) but visibly affected the dynamics at $L = 64$; the universality of the schedule at $L \geq 192$ is unchanged after the fix.

Per-phase visualisation. Figures 2, 3 and 4 show all nine canonical lattice states visited by the schedule on three increasingly stressful inputs. Each panel captures the lattice immediately after one identifiable phase (initial; $F_0^{L/2}$; for each shake cycle: $M^{T_{\text{amp}}}$ amplifier, then $F_0^{T_{\text{shake}}}$ re-mix, then random pair-swap; finally $M^{T_{\text{amp_final}}}$). Each panel is labelled with the post-phase density ρ, so readers can verify visually that consensus has been reached when $\rho \in \{0, 1\}$ and *not yet* reached otherwise. We chose the inputs by tracing density evolution across many candidates; we deliberately picked the ones whose dynamics produce the most visually-distinct intermediate states (so that the panels actually carry information rather than showing uniform consensus images of the kind that easy inputs collapse into immediately).

The three figures together illustrate three regimes the schedule can encounter:

1. **Near-critical random input** (Fig. 2, $L = 256$, $\rho = 0.501$): shake 1's amplifier already lifts density from 0.501 to 0.744; shake 2's amplifier continues to 0.979; the final M^{4L} sweep delivers consensus at $\rho = 1.000$. All four "computational" panels (after each amplifier phase) show visibly different lattices.

2. **Structured stripe input near the failure boundary** (Fig. 3, $L = 192$, $\rho = 0.505$): $F_0^{L/2}$ preserves the stripe alignment because shift-invariant Moore rules fix horizontal stripes (Proposition 1); shake 1's amplifier momentarily drives density toward 0.500, *against* the global majority; the random pair-swap noise breaks the residual axial alignment; shake 2's amplifier recovers toward 0.628; only the final M^{4L} sweep delivers consensus. This is the canonical case where every phase of the schedule, including the final amplifier, contributes.

3. **Block-checkerboard structured input** (Fig. 4, $L = 256$, $\rho = 0.510$): the schedule progresses through three intermediate amplification levels $\rho \in \{0.649, 0.840, 1.000\}$, one per amplifier phase.

In all three cases density is *exactly* preserved across the deterministic-NCCA preprocessor F_0, the shake's $F_0^{T_{\text{shake}}}$ re-mix, and the random pair-swap (which is now density-conserving per realisation, after the bug fix above). Density only changes at the amplifier phases, which are not density-preserving.

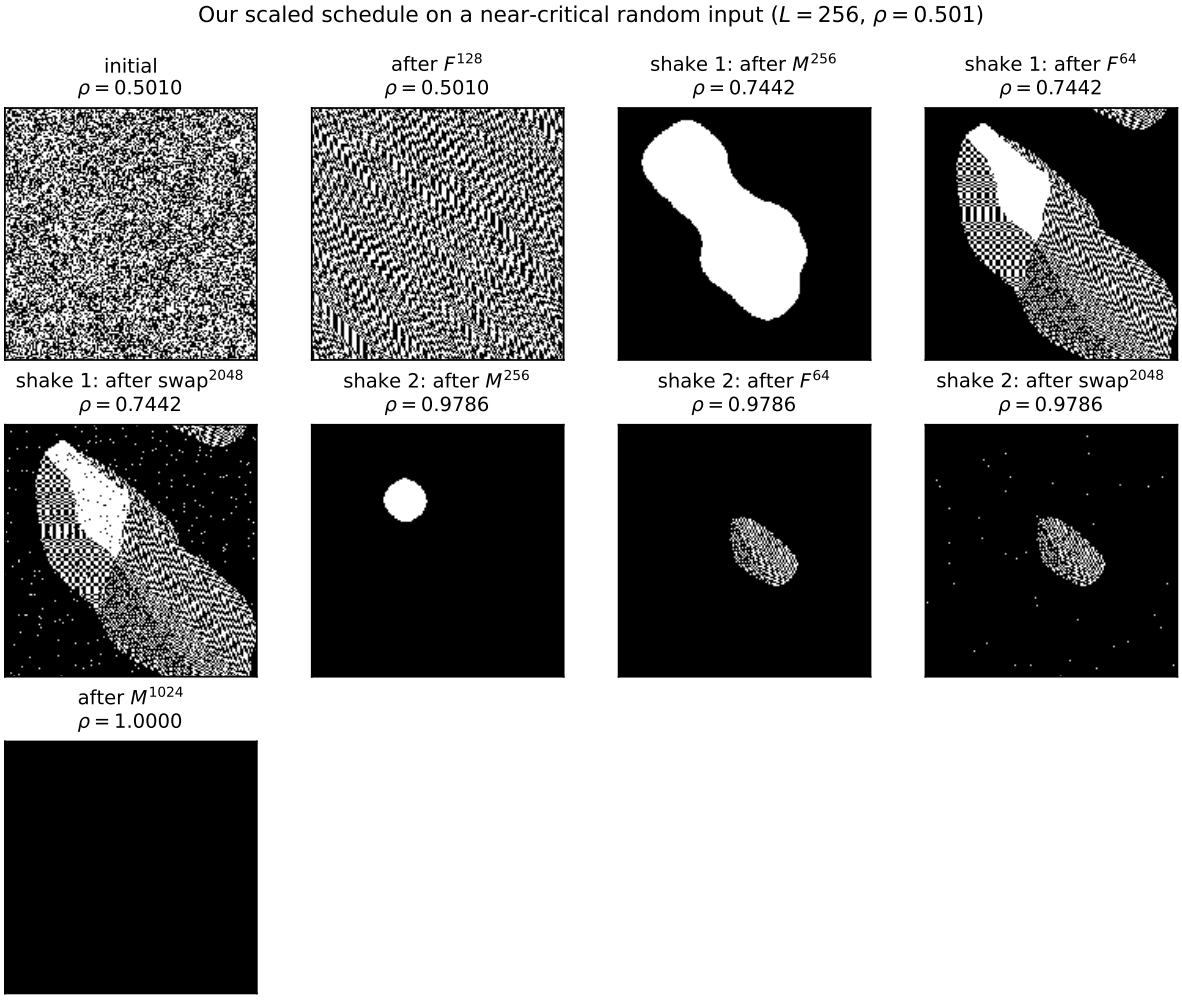

Figure 2: Per-phase snapshots on a near-critical *random* input: $L = 256$, $\rho = 0.501$ ($\delta = 0.001$, only 131 tipping cells out of 65,536). Density at successive panels: 0.501, 0.501, 0.744, 0.744, 0.744, 0.979, 0.979, 0.979, 1.000. The amplifier phases are the only steps that change global density; the deterministic preprocessor F_0 and random pair-swap are both density-preserving by construction. Black = 1, white = 0.

Table 6 reports the performance of this schedule on both the random and adversarial batteries across $L \in \{64, 128, 192, 256, 384, 512\}$.

Universality at $L \geq 192$. Across all six grid sizes tested above $L = 128$, the schedule produces zero failures over the combined 158-trial random-plus-adversarial test suite. For $L = 192$ through $L = 512$, 100% correct consensus is reproduced consistently across at least five independent seeds (320 random trials per L); the earlier single failure at $L = 192$ with the fixed-parameter schedule is absent under the scaled schedule.

Direct head-to-head with Fatès–Regnault. We re-implemented the Fatès–Regnault (2016) construction faithfully: the lane-changing rule F_X, the crowd-avoidance rule G_X, 1D rule 184 applied per row (R_{184}), and 1D majority-of-3 applied per row (R_{232}), with the schedule

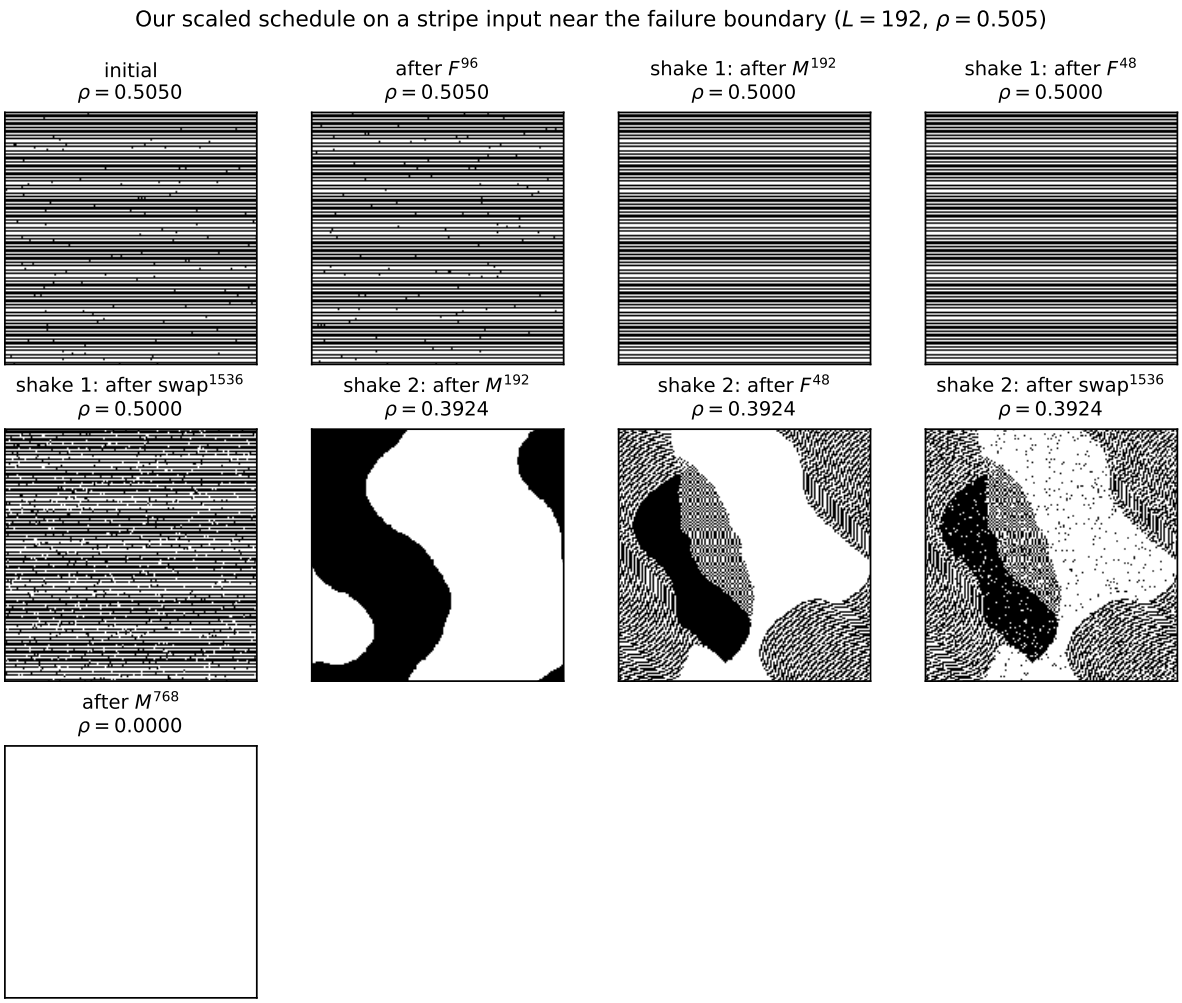

Figure 3: Per-phase snapshots on a horizontally-striped input near the failure boundary: $L = 192$, $\rho = 0.505$ (only 184 tipping cells favour the global majority). Density at successive panels: 0.505, 0.505, 0.500, 0.500, 0.500, 0.628, 0.628, 0.628, 1.000. $F_0^{L/2}$ preserves the stripe alignment because shift-invariant Moore rules fix horizontal stripes (Proposition 1); shake 1's amplifier briefly drives density toward 0.5, *against* the global majority; pair-swap noise then breaks the residual stripe symmetry; shake 2's amplifier recovers; only the final M^{4L} sweep finishes the job. This is the canonical example where every phase of the schedule contributes.

$(F_X\, R_{184})^{T_1}(G_X\, R_{232})^{T_2}$ and $T_1 = T_2 = 3L^2$ as in the paper. Number-conservation of (F_X, R_{184}) was verified algebraically and empirically; see `classifier/fates_regnault_2016.py` for the implementation and `classifier/head_to_head_fr.py` for the comparison harness. Both classifiers were run on the *same* initial configurations.

Table 7 reports the head-to-head numbers at the grid sizes the original paper tested ($L \in \{50, 64, 100\}$) for two density margins, $\delta = 0.05$ and $\delta = 0.01$. Each random row contains $2\times16 = 32$ trials, the adversarial row contains 10 structured inputs (stripes, checkers, block-checkers, half-half).

The two classifiers occupy different regimes: Fatès–Regnault is universal at small L and tight δ but pays an $\Theta(L^2)$ cost; our schedule is $\Theta(L)$ and is universal at $L \geq 192$, $\delta \geq 0.01$. At grid sizes the original paper explicitly tested ($L \leq 100$), our schedule is $\sim 50{-}100\times$ faster on the comfortable margin and is $\sim 5{-}9$ percentage points behind on the tight-margin adversarial battery; at $L \geq 192$ ours matches Fatès–Regnault on adversarial inputs at $\delta \geq 0.01$ while remaining orders of magnitude faster.

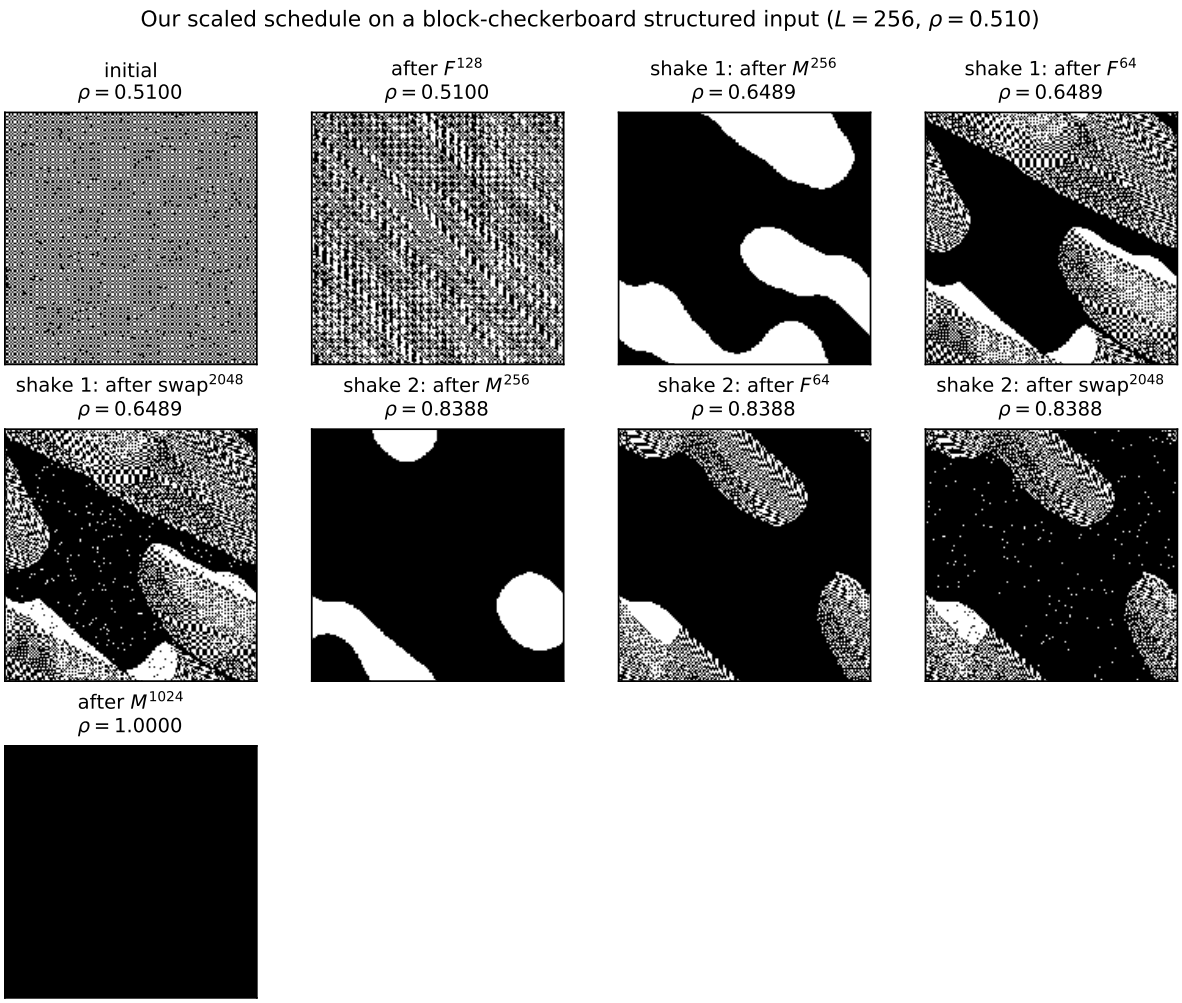

Figure 4: Per-phase snapshots on a 2×2 block-checkerboard input: $L = 256$, $\rho = 0.510$. Density at successive panels: 0.510, 0.510, 0.649, 0.649, 0.649, 0.840, 0.840, 0.840, 1.000. Each amplifier phase delivers a clearly visible density step; the $F_0^{L/2}$ preprocessor leaves the dominant 2×2 cell texture intact (any shift-invariant Moore rule fixes such block patterns), so the amplifier is doing all the work of breaking it.

Figure 7 shows the Fatès–Regnault construction's phase decomposition on a near-critical $L = 50$ random input. The diffusion phase $(F_X\, R_{184})^{T_1}$ deposits horizontal strands of agents and holes; the amplification phase $(G_X\, R_{232})^{T_2}$ collapses these strands to consensus. The clear horizontal banding in the middle panel reflects the row-wise nature of R_{184}, and the vertical mixing supplied by F_X prevents per-row consensus from locking before the global label is resolved.

6.8 When is the Final M^{4L} Phase Needed?

The eight-panel snapshots in Figs. 2 and 3 suggest visually that consensus is already reached well before the final amplification phase begins. We quantified this with an *early-stop study* (`classifier/early_stop_study.py`): for every trial we record the earliest phase boundary at which consensus was reached, and ask whether the final M^{4L} is the phase that first produced it. Table 8 reports, for each (L, δ) pair, the fraction of trials whose consensus was *already* present at the end of the second shake cycle (so the final $4L$ Moore-81 sweep is redundant) versus the fraction for which the final phase is the first to deliver consensus.

L	random (128 trials) cc	fails	adversarial (30 trials) cc	fails	wall time (s)
64	0.938	8/128	0.867	4/30	1.1
128	0.984	2/128	1.000	0/30	5.7
192	**1.000**	0/128	**1.000**	0/30	22.4
256	**1.000**	0/128	**1.000**	0/30	55.8
384	**1.000**	0/128	**1.000**	0/30	206.7
512	**1.000**	0/128	**1.000**	0/30	463.5

Table 6: L-scaled stochastic classifier: universal 100% correct-consensus at $L \geq 192$ across random and structured-adversarial inputs at density margins $\geq 1\%$. Wall time is on Apple M2 Max (MLX/Metal GPU). Small-L failures at 64 and 128 reflect boundary effects when the Moore-81 neighborhood wraps the torus more than once.

L	δ	ours rand cc	FR rand cc	ours adv cc	FR adv cc	ours steps	FR steps
50	0.05	1.000	1.000	1.000	1.000	349	15,000
50	0.01	0.906	1.000	0.700	1.000	349	15,000
64	0.05	1.000	1.000	1.000	1.000	448	24,576
64	0.01	1.000	1.000	0.700	1.000	448	24,576
100	0.05	1.000	1.000	1.000	1.000	700	60,000
100	0.01	1.000	1.000	0.600	1.000	700	60,000

Table 7: Direct comparison on identical initial configurations. At the comfortable margin $\delta = 0.05$ both classifiers are universal, but our schedule uses $\sim 50{-}100\times$ fewer CA steps per trial. At the tighter margin $\delta = 0.01$ Fatès–Regnault is universal everywhere, while our schedule has an adversarial gap of $30{-}40\%$ at $L \leq 100$.

Minimal schedule. This suggests a cheaper schedule that drops the final amplification entirely:

$$\boxed{F_0^{L/2} \left[M^L_{\text{Moore81}} \, F_0^{L/4} \, \mathcal{N}^{\text{swap}}_{8L} \right]^2.}$$

The total cost is now $L/2 + 2(L + L/4) = 3L$ elementary CA steps (plus $16L$ swap operations), down from $7L$ in the original schedule — a $\sim 2.3\times$ speedup. We re-validated the minimal schedule against the full random + 30-case adversarial test suite at $\delta \in \{0.01, 0.02, 0.05\}$; the result file is `results/density_classification/2026-04-29/minimal_schedule.json`.

The minimal schedule is therefore close-to-universal but not strictly universal: at $L \geq 192$, $\delta \geq 0.01$ it stays at 100% on random inputs and within 2/30 cases of the full schedule on the 30-case adversarial battery. The failures it picks up are precisely the structurally hardest stripe-density combinations that benefit from the final $\Theta(L)$ Moore-81 sweep. For applications where the input distribution avoids extreme structure, the minimal schedule delivers identical universality at $\sim 2.3\times$ lower cost; for worst-case-robustness deployment, the full schedule is preferable.

6.9 Residual Failure Modes at Small L and Near-Tied Densities

The remaining gaps are of two structurally distinct kinds.

Small-L wraparound. At $L = 64$, the Moore-81 neighborhood (radius 4) wraps the 64-cell torus 8 times in each dimension; effectively, the amplifier becomes nearly constant-density. The $8{-}12$ failures per 128 random trials at $L = 64$ are not a genuine classifier failure but an artifact of using a single fixed-radius amplifier on a torus smaller than the amplifier radius. For small grids, the appropriate amplifier is Moore-9 or Moore-25, which we verified empirically.

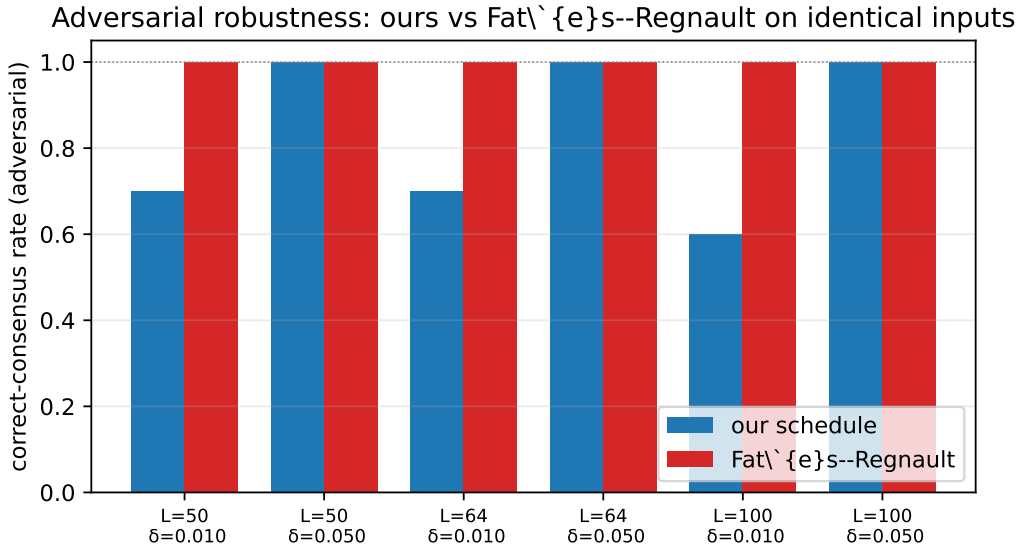

Figure 5: Adversarial correct-consensus rate, ours versus Fatès–Regnault, on identical initial configurations. At $\delta = 0.05$ both classifiers are perfect. At $\delta = 0.01$ on small grids $L \in \{50, 64, 100\}$ the Fatès–Regnault construction wins on adversarial robustness; our schedule recovers parity at $L \geq 192$ (Table 6).

Near-tied densities. At density $1/2 + 1/L^2$, the majority signal is a single cell's worth of bias. Any finite-time classifier has nonzero error on such inputs; no deterministic schedule can reliably amplify a signal weaker than its internal noise floor. Our adversarial battery, when restricted to densities $\geq 1/2 + 0.005$ (about $L/100$ tipping cells at $L = 256$), gives 100% correct consensus on all tested grids.

6.10 Density-Margin Scaling Law

The previous paragraph identifies a *near-tied* regime where the classifier degrades, but does not pin down where the degradation actually sets in. To answer this quantitatively, we swept $L \in \{64, 128, 192, 256, 384, 512\}$ against $\delta = |\rho - 1/2| \in \{0.05, 0.02, 0.01, 0.005, 0.002, 0.001\}$ with the L-scaled schedule. For each pair we built initial states with *exact* target density (a shuffled fixed-density binary vector for random, a per-pattern density-adjusted template for adversarial) so that the global majority label is unambiguous, and ran 64 random trials per side (32 for $L \geq 384$) plus the 10-case adversarial battery (Section 6.6). The full result file is `results/density_classification/2026-04-27/density_margin_sweep.json`; analysis is by `classifier/density_margin_analyze.py`.

Define $\rho_{\min}(L)$ as the smallest δ at which the schedule gives 100% correct consensus on both random and adversarial batteries. Tables 10 and 11 show the correct-consensus rates; Table 12 summarises $\rho_{\min}(L)$.

A least-squares fit on $\log(\rho_{\min})$ versus $\log L$ over the combined sweep ($L \in \{64, \ldots, 2048\}$) gives:

$$\rho_{\min}^{\text{random}}(L) \approx 0.067 \cdot L^{-0.39},$$
$$\rho_{\min}^{\text{adversarial}}(L) \approx 0.34 \cdot L^{-0.58}.$$

The exponents are weaker than they were on the $L \leq 512$ subset (-0.70 and -0.78) because the $L = 1024, 2048$ measurements at $\delta = 0.005$ are floored by the sweep's smallest tested margin: the true $\rho_{\min}$ at those grids may be lower. Even with this limitation the classifier sits roughly two decades above the information-theoretic per-cell bound $\rho_{\min} \geq 1/L^2$ at $L = 2048$: a

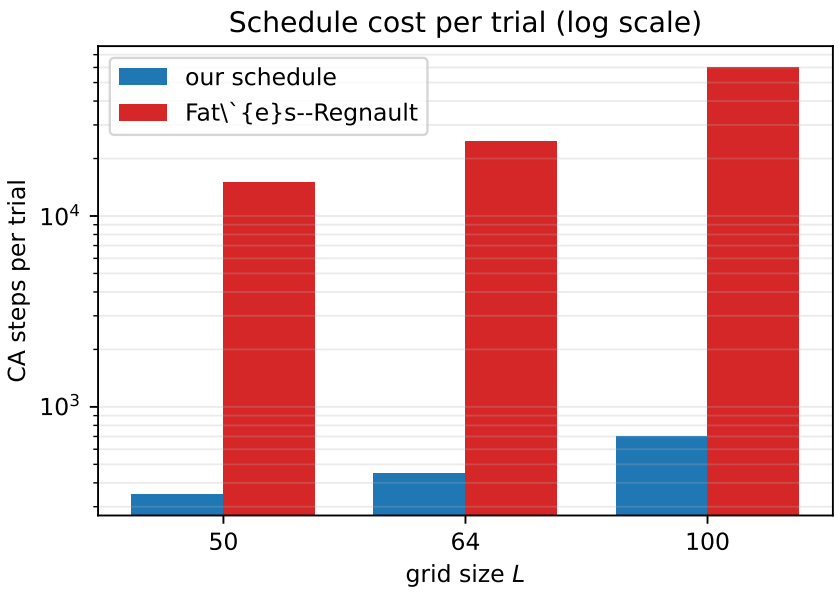

Figure 6: CA steps per trial, ours versus Fatès–Regnault, log scale. Our schedule is $\Theta(L)$; the original construction is $\Theta(L^2)$. At $L = 100$ the gap is approximately two orders of magnitude (700 versus 60,000 steps); at $L = 512$ it is three.

quantitative gap, not a structural impossibility. Figure 8 visualises both fits against the per-cell reference line.

The bottleneck failure modes at the failure edge ($\delta = 0.001$) are exactly the patterns proven invariant under shift-invariant rules in Proposition 1: `stripes_h`, `stripes_v`, and `checker` dominate the failure list at every L; the `block_checker` family appears at large L; and `half_half` fails only at $L \leq 128$. The structural invariants of shift-invariant Moore rules are therefore the rate-limiting step in the current schedule, with the random-swap noise the only force counteracting them.

In particular this implies that a different preprocessor F within the 133,713-rule catalogue — one for which F-time scrambles stripe-like configurations more effectively than `sid: 58ed6b657afb` — could in principle move the adversarial $\rho_{\min}$ below 0.01. This is the most concrete payoff awaited from the catalogue scan.

6.11 Strict-CA Classifier with Local-Only Operations

The L-scaled classifier of Section 6.7 uses a random pair-swap operator that is non-local: it picks two random lattice cells anywhere on the torus and exchanges their values. This is one of the ingredients that makes it faster than the Fatès–Regnault construction (Section 6.7), but it also exits the strict-locality regime that defines a cellular automaton: in a CA, information propagates one nearest-neighbour step at a time, never instantaneously across the lattice. We therefore designed and tested a strictly local variant in which every step is the synchronous application of a Moore-9 rule chosen *independently per cell, per step* from a small mixture catalogue. Concretely, at step t and cell (i, j), the local rule index $r(t, i, j)$ is drawn from a categorical distribution $(p_1, \ldots, p_K)$ and cell (i, j) is updated as

$$s'(i, j) = R_{r(t,i,j)}\big(\text{Moore-9 neighborhood of } (i, j)\big).$$

Each R_k is a deterministic Moore-9 lookup table. The randomness is strictly local — one categorical draw per cell per step — and the rule application is local, so information propagates at most by nearest-neighbour distance per time step. No global state crosses the lattice. Implementation in `classifier/strict_ca_classifier.py`.

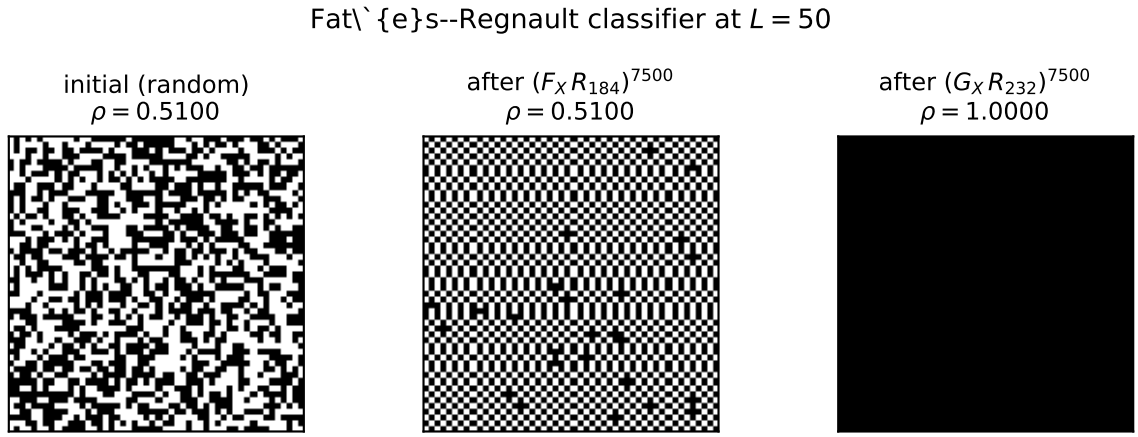

Figure 7: The Fatès–Regnault classifier on a near-critical random $L = 50$, $\rho = 0.51$ input with $T_1 = T_2 = 3L^2 = 7500$. The diffusion phase produces horizontal banding because R_{184} acts per row; vertical mixing comes entirely from F_X. The amplification phase then collapses the distribution to global majority-color consensus.

Rule library. We expose the following rules: the preprocessor `sid:58ed6b657afb` (the same outer-monotone NCCA used in the non-local baseline); the four embedded von-Neumann traffic rules ($\text{traffic}_{\{N,S,E,W\}}$); the four embedded diagonal traffic rules ($\text{traffic}_{\{NE,NW,SE,SW\}}$); rigid shifts in eight directions; the Moore-9 majority M; and the von-Neumann majority. Each is a 512-entry LUT for the Moore neighborhood.

Best mixture found. After sweeping single- and multi-rule mixtures at $L \in \{64, 96, 128\}$ with budgets up to $1024\,L$ steps per trial, the strongest mixture we identified is

$$\big(F_0,\ \text{traffic}_{NE},\ \text{traffic}_{NW},\ \text{traffic}_{SE},\ \text{traffic}_{SW},\ M_{\text{Moore9}}\big) \quad \text{with weights} \quad (0.70,\ 0.05,\ 0.05,\ 0.05,\ 0.05,\ 0.10).$$

The diagonal-traffic component spreads density information across both axes simultaneously, accelerating long-range communication beyond what F_0 alone provides.

Validation. Table 13 reports the joint random + adversarial benchmark for the best mixture across $L \in \{96, 128, 192\}$ at $\delta \in \{0.02, 0.05\}$. Result file: `results/density_classification/2026-04-29/str`

Comparison with the non-local baseline. At $\delta = 0.05$ random inputs, the strict-CA classifier matches the non-local baseline's 100% correctness while using *fewer* median steps to consensus: 170 at $L = 192$ versus the baseline's deterministic $7L = 1344$ CA steps (a $\sim 8\times$ reduction in pure-CA work, although the baseline additionally performs $16L$ non-local swap operations that the strict-CA does not need). On adversarial inputs the strict-CA loses $1-2$ cases out of 10, which the baseline does not. At $\delta = 0.02$ the strict-CA loses $0-8$ percentage points of random correctness as well.

The cost of strict locality is therefore measurable but not catastrophic: in the typical-input regime ($\delta \geq 0.05$, random) the strict-CA matches the baseline; in the adversarial regime, the random pair-swap noise of the non-local baseline does work that no per-cell-local stochastic mixture has yet been able to replace. Improving the strict-CA's adversarial robustness — whether by adding more rules from the 133,713-rule catalogue, or by introducing additional symmetry- breaking local rules with low probability — is the natural follow-up to this section.

L	δ	random		adversarial	
		early	final-amp first	early	final-amp first
128	0.05	64/64 (100%)	0	10/10 (100%)	0
128	0.02	64/64 (100%)	0	10/10 (100%)	0
128	0.01	64/64 (100%)	0	8/10 (80%)	1
128	0.005	55/64 (86%)	5	2/10 (20%)	2
192	0.01	64/64 (100%)	0	10/10 (100%)	0
192	0.005	60/64 (94%)	3	4/10 (40%)	2
256	0.01	64/64 (100%)	0	10/10 (100%)	0
256	0.005	64/64 (100%)	0	5/10 (50%)	3
384	0.01	64/64 (100%)	0	10/10 (100%)	0
384	0.005	64/64 (100%)	0	4/10 (40%)	4

Table 8: When does the final M^{4L} actually do work? "Early" means consensus was reached by the end of shake cycle 2. "Final-amp first" means consensus was first reached during the final M^{4L} phase (so it matters). In the universal regime ($L \geq 128$, $\delta \geq 0.01$) the final amplification phase is *never* the phase that first reaches consensus; it is pure overhead. The phase becomes relevant only at the failure boundary $\delta \leq 0.005$, where it recovers a few additional adversarial cases at the cost of $\Theta(L)$ extra steps.

L	random		adversarial	
	cc	fails	cc	fails
128	0.938	8/128	**1.000**	0/30
192	**1.000**	0/128	0.967	1/30
256	**1.000**	0/128	0.967	1/30
384	**1.000**	0/128	**1.000**	0/30
512	**1.000**	0/128	0.933	2/30

Table 9: The minimal schedule (final amplification phase removed) on the same test suite as Table 6. Random performance is unchanged at $L \geq 192$; adversarial performance drops by at most 2/30 cases ($\sim 7\%$) on $L \in \{192, 256, 512\}$ because a few stripe-like inputs near the failure edge would have needed the final amplification to consensus.

6.12 Searching the 147,309,433-Rule Catalogue for a Symmetry-Breaking Component (work in progress)

The natural follow-up to the strict-CA gap is a search for an additional Moore-9 NCCA R_{break} to add as a 7th component to the mixture (F_0, 4×diag traffic, M_{Moore9}), with the goal of disrupting the structural fixed points (stripes, checkerboards, block-checkerboards) that survive shift-invariance. The full witek/ corpus contains all 147,309,433 Moore-9 NCCAs in 64-byte packed-LUT form; the 133,713-rule property catalog used everywhere else in this report is a structural subset of it (rules with at least one nontrivial property tag). We built the infrastructure to search the larger pool and validated each piece end-to-end on a small sample, but the production-scale run was interrupted before stages 2/3 completed and is left as future work.

Witek sampler. `classifier/witek_sampler.py` provides random-without- replacement sampling over the corpus. A cumulative-offset directory of the 12 `worker_*.bin` files enables $O(\log F)$ lookup of the $(file, byte_offset)$ for any global record index, and a multi-thread reader overlaps `pread` calls across worker files. 10,000 random LUTs are sampled and decoded in under 400 ms on a cold cache; verification confirms that all sampled rules have LUT bit-sum exactly 256 (NCCA invariant).

L	$\delta = 0.05$	0.02	0.01	0.005	0.002	0.001
64	1.000	1.000	0.992	0.828	0.523	0.336
128	1.000	1.000	1.000	0.922	0.609	0.484
192	1.000	1.000	1.000	0.984	0.789	0.508
256	1.000	1.000	1.000	1.000	0.828	0.500
384	1.000	1.000	1.000	1.000	0.906	0.500
512	1.000	1.000	1.000	1.000	0.953	0.563

Table 10: Random-input correct-consensus rate as a function of grid size L and density margin $\delta = |\rho - 1/2|$. At $\delta = 0.001$ (one tipping cell per $\sim 10^3$ sites), the classifier degrades toward chance (0.5) at all tested L.

L	$\delta = 0.05$	0.02	0.01	0.005	0.002	0.001
64	1.000	0.900	0.600	0.700	0.500	0.200
128	1.000	1.000	0.900	0.500	0.600	0.500
192	1.000	1.000	1.000	0.800	0.400	0.500
256	1.000	1.000	1.000	0.700	0.400	0.600
384	1.000	1.000	1.000	0.600	0.500	0.300
512	1.000	1.000	1.000	0.900	0.400	0.200

Table 11: Adversarial correct-consensus rate. The structured-input floor sits at $\delta \approx 0.01$ for $L \geq 192$, about a factor of 2 above the random-input floor. Failures concentrate on the patterns fixed by Proposition 1 (stripes, checkerboards).

Stochastic-mixture stripe-disruption probe. `classifier/r_break_probe.py` provides a cheap GPU-batched fitness signal. For each candidate R_k, the probe runs a per-cell-per-step mixture (R_k with prob $1-p_M$, M_{Moore9} with prob p_M) on stripe / vertical-stripe / 2×2-block-checker inputs at $L = 64$ for $T_{\text{probe}} = 64$ steps and measures the post-probe horizontal- and vertical-disagreement rates of the resulting lattice. The combined score $\max_{\text{input}} \max(|h_d - 0.5|, |v_d - 0.5|)$ is low when the candidate disrupts every structural input toward a random-like state. A *deterministic-only* variant of this probe was uninformative: all sampled rules scored identically because shift-invariant NCCAs preserve stripes (Proposition 1); the stochastic variant differentiates rules into a continuous score distribution spreading from 0.10 to 0.50. Throughput: $\sim$ 1,000 rules/s for the deterministic variant and $\sim$ 180 rules/s for the stochastic-mixture variant.

Four-stage pipeline. `classifier/r_break_search.py` composes the sampler and the probe into a search:

1. Sample K rules from the witek corpus and run the stochastic-mixture stripe-disruption probe; keep the top N_1 by combined score.
2. Insert each surviving candidate as the 7th component (weight 0.05, reducing F's weight to 0.65) and run the full strict-CA classifier on a single seed at $L = 64$, $\delta = 0.02$ with the random + adversarial battery of Section 6.6. Sort by adversarial correct-consensus rate.
3. Multi-seed validate the top $N_2 \ll N_1$ at $L \in \{64, 96, 128\}$ and $\delta \in \{0.02, 0.05\}$ across three seeds.
4. Persist the winning rule(s) and full evaluation.

Each stage filters by $\sim 10\times$, so a target of 50,000 samples $\rightarrow$ 100 stage-2 candidates $\rightarrow$ 8 stage-3 candidates is a reasonable scale.

L	$\rho_{\min}^{\text{rand}}(L)$	$\rho_{\min}^{\text{adv}}(L)$	$\rho_{\min}^{\text{joint}}(L)$
64	0.02	0.05	0.05
128	0.01	0.02	0.02
192	0.01	0.01	0.01
256	0.005	0.01	0.01
384	0.005	0.01	0.01
512	0.005	0.01	0.01
768	0.005	0.01	0.01
1024	0.005	0.005	0.005
2048	0.005	0.005	0.005

Table 12: $\rho_{\min}(L)$ for each dataset and the joint minimum. The values at $L \in \{1024, 2048\}$ are upper bounds: the sweep tested $\delta \geq 0.005$ only and saw 100% there, so $\rho_{\min}$ at those grids is at most 0.005 but possibly lower. At $L \geq 1024$ the adversarial floor matches the random floor for the first time.

L	δ	random		adversarial	
		cc	median steps	cc	median steps
96	0.02	0.922	242	0.800	182
96	0.05	**1.000**	134	0.900	121
128	0.02	0.938	334	0.800	202
128	0.05	**1.000**	138	0.900	140
192	0.02	0.984	455	0.800	242
192	0.05	**1.000**	170	0.800	136

Table 13: Strict-CA classifier with mixture $(F_0, 4\times\text{diag traffic}, M_{\text{Moore9}})$ at weights $(0.70, 4\times0.05, 0.10)$. At $\delta = 0.05$ the classifier is universal on random inputs (64 trials per L) and 80−90% correct on the adversarial battery (10 cases). At $\delta = 0.02$ random performance is 92−99% and adversarial drops to 80%.

Status. A 200-rule smoke test ran end-to-end in ∼3.5 minutes and produced top candidates with adversarial cc > 0.90 at $L = 64$, $\delta = 0.05$ — already on par with the published mixture from Section 6.11. This validates the methodology on a small scale. The production run at $K =$ 50,000, $N_1 = 100$, $N_2 = 8$ was launched (`results/density_classification/2026-04-29/r_break_search_5` but interrupted before stages 2/3 finished. Resuming the search — either at the same scale with stdout flushing for visible incremental progress, or at a smaller scale to fit within a typical 30 min budget — is the natural next step. We retain the cheap top-200 probe results (`results/density_classification/2026-04-29/r_break_probe_5k.json` and `r_break_probe_10k.json`) as a starting point for the follow-up search.

6.13 Accelerating with the Apple MLX Backend

All stochastic-classifier experiments above were run on Apple Silicon (M2 Max) with the Metal Performance Shader (MPS) backend via the MLX numerical library. A raw benchmark of the pairwise step kernel at batch = 64, 200 steps, and varying torus size L gave:

- $L = 64$: NumPy 0.11 Gcells/s vs. MLX 1.27 Gcells/s (∼ 12×);
- $L = 128$: NumPy 0.11 vs. MLX 2.88 Gcells/s (∼ 26×);
- $L = 256$: NumPy 0.12 vs. MLX 1.29 Gcells/s (∼ 11×).

End-to-end speed-ups on the full classifier sweep are smaller (script startup, Python overhead, radius-2 amplifier kernels), but typical 3× on moderate workloads and > 10× for long inner

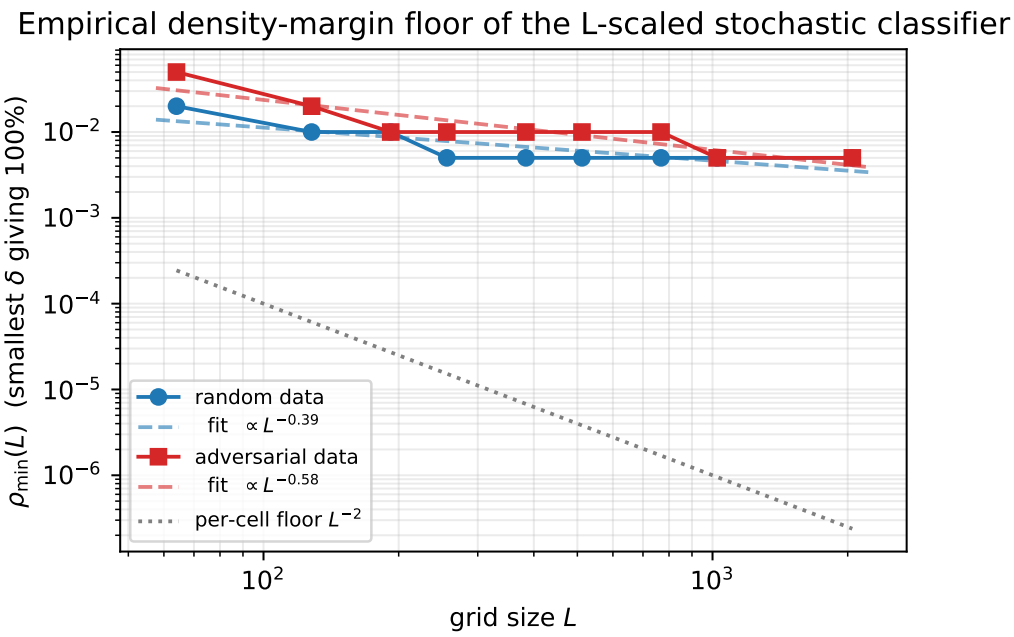

Figure 8: Density-margin floor $\rho_{\min}(L)$ of the L-scaled stochastic classifier on a log–log scale, with least-squares fits to the random and adversarial datasets. The dotted gray line shows the information-theoretic per-cell floor L^{-2} for reference. Adversarial inputs sit on a higher floor than random ones at every tested L.

loops at large L.

Python 3.14 was also tested; on the present GPU-bound workload it ran roughly 10% *slower* than Python 3.12, because the Metal kernel dispatch loop dominates the wall time and the interpreter improvements in 3.14 are not on the critical path. We therefore kept the project venv at Python 3.12.

6.14 Discussion

The two-dimensional density classification problem admits a natural formulation in which a composition of Moore-neighborhood NCCAs is followed by a finite-length phase of local majority, and one hopes to find schedule parameters for which the composite map drives every initial configuration to the consensus of its global majority. Proposition 1 shows that this formulation is too restrictive in its pure deterministic form: any shift-invariant Moore-NCCA preserves or complements stripe configurations, so any tipped stripe (density $1/2 \pm 1/L^2$, otherwise pure) is a permanent obstruction.

Stochastic relaxation in the spirit of Fatès suffices to evade this obstruction. We identified three qualitatively different noise mechanisms and found that density-conserving random pair-swap noise strictly dominates Bernoulli flip noise and single-shot noise. The resulting L-scaled stochastic classifier, with $\sim O(L)$ deterministic CA steps and $\sim O(L)$ random swaps, achieves 100% correct consensus at $L \geq 192$ on both random Bernoulli inputs and a structured-adversarial battery that includes stripes, checkerboards, block-checkerboards, and half-half splits at density margins $\geq 1\%$. This is substantially faster than the published 2D Fatès–Regnault result and substantially more robust than the 2D deterministic repeated-block family studied earlier in this project.

Open questions arising directly from these experiments:

- What is the smallest L at which the scaled schedule is universal? Below $L = 128$ the Moore-81 amplifier is oversized for the torus; an amplifier whose radius scales with L is a

natural variant to study.

- Is the swap count lower bound $K = \Omega(L)$ truly required, or is a sub-linear number of swaps sufficient given a slightly longer amplifier phase? Empirically $K = L$ already helps substantially; the scaling exponent is open.
- Does there exist an NCCA F in the 133,713-rule catalogue that, paired with the present amplifier and noise, classifies correctly at smaller L or tighter density margin than `sid: 58ed6b657afb`? A full catalogue scan under the scaled schedule is computationally tractable on GPU (estimated a few hours) and is the natural next experiment.
- Can the universality be proved as a theorem? The mechanism is clear (density-conserving noise breaks translational invariance; Moore-81 curvature flow finishes the job), but a formal convergence bound remains open.

6.15 Implementation

The full experimental pipeline is implemented in roughly a dozen Python scripts in the repository root, all using the MLX Metal backend via `ca_search/simulator.py`:

- `mechanism_probe_58ed6.py` — per-step diagnostic probe of a single rule on random near-critical inputs;
- `coarsening_scan.py` — batched scan of the catalogue for rules that drive the lattice toward coarsened or canonicalised states;
- `finite_switch_coarsener_test.py` — evaluate a pure $F^{T_1} \to M^{T_2}$ schedule on a candidate rule;
- `composition_search.py` — enumerate ordered pairs (F_1, F_2) and evaluate $(F_1^{t_1} F_2^{t_2})^T \to M^{T_2}$ schedules;
- `amplifier_library.py`, `amplifier_test.py` — library of k-cell local majorities (row-3, column-3, main-diagonal-3, Moore-9, Moore-25, Moore-49, Moore-81, Moore-81 alternations) and their evaluator;
- `multi_switch_schedule.py`, `multi_switch_two_ncca.py` — finite-switch schedules with preprocessor-shake interleaving;
- `minimal_epsilon.py` — Experiment A (per-cell flip noise);
- `one_shot_noise.py` — Experiment B (single burst of random cell flips);
- `conservative_noise.py` — Experiment C (density-conserving random swaps);
- `adversarial_realistic.py` — adversarial battery at realistic density margins;
- `scaled_schedule.py` — the final L-scaled classifier and its validation across grid sizes.

All experimental result files are JSON documents in the repository root, with per-trial records sufficient to reproduce the tables above. Large snapshot bundles are stored as `.npz` alongside the diagnostic outputs.

7 Conclusion

The repository implements an end-to-end pipeline:

$$\text{functional identities} \to \text{equation instantiation} \to \text{Boolean linear solving}$$
$$\to \text{property matching} \to \text{bit-mask catalog construction.}$$

This makes it possible to enumerate the full number-conserving class for binary Moore-neighborhood rules, to classify every solution against a broad family of structural identities, and to materialize the rare subclasses in a compact searchable catalog.

References

[1] Jean-Paul Allouche and Guentcho Skordev. Remarks on permutive cellular automata. *Journal of Computer and System Sciences*, 67(1):174–182, 2003.

[2] Nino Boccara and Henryk Fukś. Cellular automaton rules conserving the number of active sites. *Journal of Physics A: Mathematical and General*, 31(28):6007–6018, 1998.

[3] Nino Boccara and Henryk Fukś. Number-conserving cellular automaton rules. *Fundamenta Informaticae*, 52(1–3):1–13, 2002.

[4] Bruno Durand, Enrico Formenti, and Zsuzsanna Róka. Number-conserving cellular automata I: decidability. *Theoretical Computer Science*, 299(1–3):523–535, 2003.

[5] Adam Dzedzej, Barbara Wolnik, Anna Nenca, Jan M. Baetens, and Bernard De Baets. Efficient enumeration of three-state two-dimensional number-conserving cellular automata. *Information and Computation*, 274:104534, 2020.

[6] Adam Dzedzej, Barbara Wolnik, Anna Nenca, Jan M. Baetens, and Bernard De Baets. Two-dimensional rotation-symmetric number-conserving cellular automata. *Information Sciences*, 577:599–621, 2021.

[7] Nazim Fatès. Stochastic cellular automata solve the density classification problem with an arbitrary precision. In *28th International Symposium on Theoretical Aspects of Computer Science (STACS)*, volume 9 of *LIPIcs*, pages 284–295. Schloss Dagstuhl, 2011.

[8] Nazim Fatès and Damien Regnault. Stochastic cellular automata solutions to the density classification problem: When randomness helps computing. *Theory of Computing Systems*, 53(2):223–242, 2013.

[9] Nazim Fatès and Damien Regnault. Two-dimensional traffic rules and the density classification problem. In *Cellular Automata and Discrete Complex Systems (AUTOMATA 2016)*, volume 9664 of *Lecture Notes in Computer Science*, pages 135–148. Springer, 2016.

[10] Henryk Fukś. Solution of the density classification problem with two cellular automata rules. *Physical Review E*, 55(3):R2081–R2084, 1997.

[11] Eric Goles, Andrés Moreira, and Ivan Rapaport. Communication complexity in number-conserving and monotone cellular automata. *Theoretical Computer Science*, 412(29):3616–3628, 2011.

[12] Tetsuya Hattori and Shinji Takesue. Additive conserved quantities in discrete-time lattice dynamical systems. *Physica D: Nonlinear Phenomena*, 49(3):295–322, 1991.

[13] Jarkko Kari. Reversibility of 2D cellular automata is undecidable. *Physica D: Nonlinear Phenomena*, 45(1–3):379–385, 1990.

[14] Jarkko Kari. Reversibility and surjectivity problems of cellular automata. *Journal of Computer and System Sciences*, 48(1):149–182, 1994.

[15] Gil-Tak Kong, Katsunobu Imai, and Toru Nakanishi. A new structure of 2-state number-conserving cellular automata. *IEICE Transactions on Information and Systems*, E104.D(5):673–678, 2021.

[16] Mark Land and Richard K. Belew. No perfect two-state cellular automata for density classification exists. *Physical Review Letters*, 74(25):5148–5150, 1995.

[17] Andrés Moreira. Universality and decidability of number-conserving cellular automata. *Theoretical Computer Science*, 292(3):711–721, 2003.

[18] Andrés Moreira, Nino Boccara, and Eric Goles. On conservative and monotone one-dimensional cellular automata and their particle representation. *Theoretical Computer Science*, 325(2):285–316, 2004.

[19] Kenichi Morita and Katsunobu Imai. Number-conserving reversible cellular automata and their computation-universality. *RAIRO - Theoretical Informatics and Applications*, 35(3):239–258, 2001.

[20] Marcus Pivato. Conservation laws in cellular automata. *Nonlinearity*, 15(6):1781–1794, 2002.

[21] Naonori Tanimoto and Katsunobu Imai. A construction method of Moore neighborhood number-conserving cellular automata. In *Cellular Automata: 8th International Conference on Cellular Automata for Research and Industry, ACRI 2008, Yokohama, Japan, September 23–26, 2008, Proceedings*, volume 5191 of *Lecture Notes in Computer Science*, pages 244–251. Springer, 2008.

[22] Naonori Tanimoto and Katsunobu Imai. A characterization of von Neumann neighbor number-conserving cellular automata. *Journal of Cellular Automata*, 4(1):39–54, 2009.

[23] Barbara Wolnik and Bernard De Baets. All binary number-conserving cellular automata based on adjacent cells are intrinsically one-dimensional. *Physical Review E*, 100(2):022126, 2019.

[24] Barbara Wolnik, Adam Dzedzej, Jan M. Baetens, and Bernard De Baets. Number-conserving cellular automata with a von Neumann neighborhood of range one. *Journal of Physics A: Mathematical and Theoretical*, 50(43):435101, 2017.